CLAUDE CERTIFIED ARCHITECT - FOUNDATIONS | EXAM CODE CCAR-F

Mock Examination 01

Claude Certified Architect - Foundations
60 scenario-based questions | 120 minutes

Scenarios assessed in this paper: Scenario 1 - Customer Support Resolution Agent; Scenario 2 - Code Generation with Claude Code; Scenario 3 - Multi-Agent Research System; Scenario 4 - Developer Productivity with Claude

Format: multiple-choice and multiple-response; every question states how many answers to select.

Contents: instructions and blueprint | candidate answer sheet | 60 questions | quick answer key and score tracker | detailed explanations for every option | domain-level review guide | revision checklist.

Status: ORIGINAL PRACTICE MATERIAL. This is not an official Anthropic examination paper and contains no confidential exam content. It was written against the publicly available Claude Certified Architect - Foundations Exam Guide (v1.0, effective July 2026).

Contents

1. Instructions and exam conditions 3

2. What is verified, what is supplied, and what is original 3

3. Exam blueprint for this paper 4

Candidate answer sheet 5

4. Examination - Questions 1 to 60 6

Scenario 1: Customer Support Resolution Agent (Questions 1-17) . . . 6

Scenario 2: Code Generation with Claude Code (Questions 18-31) . . . 11

Scenario 3: Multi-Agent Research System (Questions 32-47) . . . 14

Scenario 4: Developer Productivity with Claude (Questions 48-60) . . . 18

5. Quick answer key and score tracker 22

6. Detailed answer key and explanations 23

7. Domain-level performance review guide 50

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here) . . . 50

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here) . . . 50

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here) . . . 50

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here) . . . 50

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here) . . . 51

8. Final revision checklist 52

1. Instructions and exam conditions

Time limit: 120 minutes for 60 questions (an average of 2 minutes per question). Work in one sitting, without notes, documentation, or AI assistance, to get a realistic read on your readiness.

• Every question is anchored to one of the production scenarios in this paper. Read the scenario description at the start of each scenario section; questions are otherwise self-contained.
• Each question states **how many answers to select**. For 'Select ONE', exactly one option is best. For 'Select exactly TWO/THREE', all selected options must be correct; there is no partial credit in this practice format.
• Choose the *best* answer for the stated constraints, not an answer that could merely work. Where a question asks for the FIRST, SAFEST, or MOST RELIABLE step, that qualifier is part of the question.
• Attempt every question. Unanswered questions score zero. There is no 'all of the above' or 'none of the above'.
• Do not look at the answer key until you have finished. The key, explanations, and review guide begin after Question 60.

2. What is verified, what is supplied, and what is original

Category	What it covers
Verified official information	From the official Exam Guide (CCAR-F v1.0): 60 items; multiple-choice and multiple-response items that state how many responses to select; 4 scenarios drawn from a bank of 6; 120-minute limit; scaled passing score of 720 on a 100-1,000 scale; score report with percent-correct by domain; domain weights 27/18/20/20/15%; the 31 task statements and the in-scope/out-of-scope lists.
Scenario descriptions	The six production scenarios are quoted from the official guide; the extra topic lists you were given (e.g., idempotency, structured handoffs, schema design) guided question themes. Scenario text shown above each question group is the official description.
Original practice content	All questions, options, explanations, and review guidance in this file were written for this pack. They are not official questions, are not copied from commercial practice exams or leaked banks, and may differ in difficulty from the real exam.
Supplemental knowledge	A few items use general engineering knowledge to make a scenario realistic (for example CI/CD terminology or least-privilege shell access). They are flagged 'Supplemental' in the explanation and are not confirmed exam objectives.

A raw percentage on this paper is **not** a scaled score. The real exam reports a scaled score (100-1,000) with a cut score of 720 derived from a formal standard-setting study; percent-correct by domain is shown for information only. Doing well on these papers does not guarantee a pass.

3. Exam blueprint for this paper

Domain	Official weight	Questions in this paper
1. Agentic Architecture & Orchestration	27%	16
2. Tool Design & MCP Integration	18%	11
3. Claude Code Configuration & Workflows	20%	12
4. Prompt Engineering & Structured Output	20%	12
5. Context Management & Reliability	15%	9
Total	**100%**	**60**

Question counts are the closest integer allocation of the official weights to 60 questions. Each question has exactly one primary domain; secondary domains appear in the answer key where a question also touches another domain.

Scenario	Questions	Numbers
Scenario 1 - Customer Support Resolution Agent	17	1-17
Scenario 2 - Code Generation with Claude Code	14	18-31
Scenario 3 - Multi-Agent Research System	16	32-47
Scenario 4 - Developer Productivity with Claude	13	48-60

Design targets (not claims about the real exam): about 20% foundational, 55% intermediate, 25% challenging. In this paper: 13 foundational, 32 intermediate, 15 challenging. Response types: 54 single-answer, 5 select-two, 1 select-three.

Candidate answer sheet

Mark your answers here while taking the paper. Cells that do not exist for a question are greyed out. Where a question says 'Select exactly TWO/THREE', mark exactly that many letters. Do not turn to the answer key until you have finished all 60 questions (or the 120 minutes have elapsed).

Name: _____ Date: _____ Start time: _____ End time: _____

No.	Pick	A	B	C	D	E	F
1	1	(A)	(B)	(C)	(D)		
2	1	(A)	(B)	(C)	(D)		
3	1	(A)	(B)	(C)	(D)		
4	1	(A)	(B)	(C)	(D)		
5	1	(A)	(B)	(C)	(D)		
6	1	(A)	(B)	(C)	(D)		
7	1	(A)	(B)	(C)	(D)		
8	1	(A)	(B)	(C)	(D)		
9	1	(A)	(B)	(C)	(D)		
10	1	(A)	(B)	(C)	(D)		
11	1	(A)	(B)	(C)	(D)		
12	1	(A)	(B)	(C)	(D)		
13	2	(A)	(B)	(C)	(D)	(E)	
14	1	(A)	(B)	(C)	(D)		
15	3	(A)	(B)	(C)	(D)	(E)	(F)
16	1	(A)	(B)	(C)	(D)		
17	1	(A)	(B)	(C)	(D)		
18	1	(A)	(B)	(C)	(D)		
19	1	(A)	(B)	(C)	(D)		
20	2	(A)	(B)	(C)	(D)	(E)	
21	1	(A)	(B)	(C)	(D)		
22	2	(A)	(B)	(C)	(D)	(E)	
23	1	(A)	(B)	(C)	(D)		
24	2	(A)	(B)	(C)	(D)	(E)	
25	1	(A)	(B)	(C)	(D)		
26	1	(A)	(B)	(C)	(D)		
27	1	(A)	(B)	(C)	(D)		
28	1	(A)	(B)	(C)	(D)		
29	1	(A)	(B)	(C)	(D)		
30	1	(A)	(B)	(C)	(D)		

No.	Pick	A	B	C	D	E	F
31	1	(A)	(B)	(C)	(D)		
32	1	(A)	(B)	(C)	(D)		
33	1	(A)	(B)	(C)	(D)		
34	1	(A)	(B)	(C)	(D)		
35	1	(A)	(B)	(C)	(D)		
36	1	(A)	(B)	(C)	(D)		
37	1	(A)	(B)	(C)	(D)		
38	1	(A)	(B)	(C)	(D)		
39	1	(A)	(B)	(C)	(D)		
40	1	(A)	(B)	(C)	(D)		
41	1	(A)	(B)	(C)	(D)		
42	1	(A)	(B)	(C)	(D)		
43	1	(A)	(B)	(C)	(D)		
44	1	(A)	(B)	(C)	(D)		
45	1	(A)	(B)	(C)	(D)		
46	1	(A)	(B)	(C)	(D)		
47	1	(A)	(B)	(C)	(D)		
48	1	(A)	(B)	(C)	(D)		
49	1	(A)	(B)	(C)	(D)		
50	1	(A)	(B)	(C)	(D)		
51	1	(A)	(B)	(C)	(D)		
52	1	(A)	(B)	(C)	(D)		
53	1	(A)	(B)	(C)	(D)		
54	1	(A)	(B)	(C)	(D)		
55	1	(A)	(B)	(C)	(D)		
56	1	(A)	(B)	(C)	(D)		
57	2	(A)	(B)	(C)	(D)	(E)	
58	1	(A)	(B)	(C)	(D)		
59	1	(A)	(B)	(C)	(D)		
60	1	(A)	(B)	(C)	(D)		

4. Examination - Questions 1 to 60

Time allowed: 120 minutes. Start the clock now.

Scenario 1: Customer Support Resolution Agent (Questions 1-17)

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Question 1

Select ONE answer.

A support agent resumes a named session two days later when a customer returns about order #55120. The saved history contains a lookup_order result showing the order as 'processing'. Since then the warehouse shipped it, but the resumed agent tells the customer the order has not shipped yet. The reply is well formed and polite, but it is wrong. What is the most reliable way to prevent this?

A. Start a new session seeded with a structured summary of confirmed case facts, and call lookup_order again for current order state.

B. Fork the session with fork_session and ask the forked branch to answer from the saved analysis without further tool calls.

C. Switch to a model with a larger context window so that the newest user message outweighs older tool output in the history.

D. Resume the same session and add a system prompt line saying that earlier tool results may be outdated and should be treated cautiously.

Question 2

Select ONE answer.

Logs show the agent correctly calls get_customer and receives verified customer ID C-1042. The caller then asks for a refund on order #88231, and the agent calls process_refund for it. An audit later finds order #88231 belongs to customer C-7710. The team already has a hook that blocks process_refund until get_customer has succeeded in the session. What change best closes this gap?

A. In a PreToolUse hook for process_refund, load the order and reject the call unless its customer ID equals the verified customer ID for the session.

B. Ask the caller to restate the email address on the account before each refund and compare it to the get_customer record in the prompt.

C. Update the system prompt to tell the agent that it must confirm an order belongs to the verified customer before issuing a refund.

D. Extend the existing prerequisite so it also requires that lookup_order has been called earlier in the session for the same order number before process_refund is allowed to run.

Question 3

Select ONE answer.

A developer wrote the agent loop to stop as soon as the assistant message contains any text block, and also caps the loop at 5 iterations. Customers sometimes see only 'Let me check that order for you' and nothing happens. Some complex billing disputes end abruptly after the fifth iteration with no resolution. Which redesign best fixes both symptoms?

A. Keep the text check, but instruct the model never to write any text before calling a tool, so that the first text block it produces always signals the final answer to the customer.

B. Continue while stop_reason is 'tool_use', run the tools and append their results, finish on 'end_turn', and keep an iteration limit only as an escalating backstop.

C. Replace the loop with a fixed sequence of get_customer, lookup_order, then process_refund so each request runs the same three steps.

D. Raise the iteration cap to 25 and also stop the loop when the response includes a phrase such as 'I have completed your request'.

Question 4

Select ONE answer.

The agent evaluates a 30-day return window using data from three MCP tools: get_customer returns account creation as a Unix timestamp, lookup_order returns ISO 8601 dates, and a legacy billing tool returns payment status as numeric codes (1, 2, 3). The agent sometimes miscalculates eligibility and misreads billing status. What is the most robust fix?

A. Describe the conversion formula for Unix timestamps and the meaning of each numeric status code in the system prompt, and ask the agent to convert every value before it reasons about eligibility.

B. Lengthen each tool's description to explain the format of its output fields so the model knows how to interpret each response.

C. Add a convert_timestamp tool and instruct the agent to call it whenever a date appears in any tool result.

D. Add a PostToolUse hook that converts all dates to one format and maps numeric status codes to labeled values before the model sees any tool result.

Question 5

Select ONE answer.

A customer writes one message: they want to return a blender from order #4410, dispute a $39 charge that appears tied to the same order, and update a shipping address. The return refund affects the amount that is actually in dispute. Which design resolves the message quickly while avoiding duplicate or conflicting refunds?

A. Process the concerns strictly in the order mentioned, starting each from a clean context so earlier issues cannot influence later ones.

B. Launch three isolated subagents that each handle one concern end to end, including calling process_refund if their analysis supports it.

C. Escalate the entire message to a human agent because it contains several issues and one of them involves money.

D. Split the message into three concerns, run read-only investigation for them in parallel from shared verified-customer and order facts, reconcile the findings, then execute any refund once.

Question 6

Select ONE answer.

The agent calls escalate_to_human with the note: 'Customer is upset, please help.' Human agents work in a separate ticketing system and cannot see the agent's conversation transcript. They must ask the customer to start over. What should the agent include to fix this?

A. A structured handoff summary with customer ID, root cause, refund amount involved, and the recommended next action.

B. Only the customer's name, with an instruction for the human agent to find the case in the CRM by searching that name.

C. A message asking the customer to describe the problem again to the human agent when the connection is made.

D. The full raw conversation text and every tool output pasted into the escalation note as one large block, so the human agent has complete information available to search through.

Question 7

Select ONE answer.

A support lead wants to compare two resolutions for a complex disputed order, a full refund versus store credit plus a partial refund. Both should start from the same completed investigation of the order history. A prototype lets both branches call process_refund while exploring. What approach is best?

A. Run strategy one in the current session and then instruct the agent to disregard it and try strategy two in the same session.

B. Fork the session and let both branches execute their own resolution against the customer's order, then issue a compensating adjustment to reverse whichever one the lead does not choose.

C. Use fork_session for two branches from the shared investigation; each drafts a resolution without executing refunds, and only the chosen resolution is executed, once.

D. Resume the same named session in two terminals with --resume and let each terminal pursue a different resolution.

Question 8

Select ONE answer.

After a prompt update, the agent started calling lookup_invoice for nearly every message that mentions money, including 'I was charged for an order that never arrived', and never checks delivery status. The new system prompt contains the rule: 'For any message mentioning a charge, payment, or price, use lookup_invoice.' Tool descriptions for lookup_invoice and lookup_order are both clear and distinct. What is the best fix?

A. Remove the keyword-triggered routing rule from the system prompt and rely on the tool descriptions, which already state each tool's inputs and boundaries.

B. Merge lookup_invoice and lookup_order into a single lookup_record tool so there is no routing decision to make.

C. Set tool_choice to force lookup_invoice on every turn so the behavior at least becomes predictable for billing messages.

D. Add more keywords such as 'shipped', 'delivery', and 'tracking' to the prompt rule and map them to lookup_order, so that delivery questions are routed to the right tool.

Question 9

Select ONE answer.

Your support-engineering team uses Claude Code while developing against the company's MCP servers for get_customer, lookup_order, process_refund, and escalate_to_human. Every engineer needs these servers when they clone the repository, but the CRM token must not be committed. One engineer also wants to try an experimental MCP server that only they will use. How should you configure this?

A. Commit .mcp.json with the literal token because the repository is private, and tell the experimenting engineer to edit the same file locally.

B. Write the shared server definitions and the literal token in CLAUDE.md so Claude Code reads them at the start of each session.

C. Put the shared servers in the project's .mcp.json using ${CRM_API_TOKEN} environment variable expansion, and add the experimental server to the engineer's user-scoped ~/.claude.json.

D. Ask each engineer to add all servers, including the shared ones, to their own ~/.claude.json using a token from the team password manager.

Question 10

Select ONE answer.

To avoid creating multiple agents, a team gave its single support agent 18 tools: the four support tools plus administrative tools such as delete_account, adjust_price_list, bulk_credit, and export_customers. In testing the agent sometimes picks a similar administrative tool for ordinary requests, and administrative actions need staff approval anyway. What is the best change?

A. Set tool_choice to 'any' so the model must call one of the 18 tools on every turn, which reduces conversational mistakes.

B. Add a second model that reviews each tool call and vetoes administrative calls, while keeping the full tool list for the main agent.

C. Keep all 18 tools and add a prompt paragraph explaining which tools are reserved for staff, with a firm warning that the agent must never call them even when a customer asks directly.

D. Give the resolution agent only the four support tools; keep administrative capabilities out of its tool set and reachable only through escalate_to_human and staff tooling.

Question 11

Select ONE answer.

process_refund currently returns the string 'Operation failed' for three different situations: a gateway timeout, an order ID that does not exist, and a refund amount above the agent's authority. The agent retries all three and sometimes tells customers 'something went wrong on our side' even when the refund exceeds policy. What is the best improvement to the tool?

A. Return a verbose stack trace and server log excerpt so the model can work out the cause by reading it.

B. Return an MCP error result (isError) with structured metadata: an error category, an isRetryable flag, and a customer-appropriate description.

C. Return an empty result for all failures so that the agent treats the refund as pending and tries again later in the conversation.

D. Keep the generic message but add a system prompt rule telling the agent to retry any failed refund up to three times with a short delay, and to escalate if it still fails.

Question 12

Select ONE answer.

To improve ambiguous-case handling, the team added eight few-shot examples to the system prompt, all of them standard damaged-item refunds with photo evidence. Now the agent also refunds cases that only look similar, such as damage reported 90 days after delivery under a 30-day policy, instead of escalating. Which change best addresses this?

A. Add twenty more standard damaged-item examples with photo evidence so the agent sees the refund pattern more often and recognizes eligible cases with higher confidence.

B. Remove the examples and add the instruction 'when in doubt, approve the refund to keep customers satisfied'.

C. Replace them with a few targeted examples near the decision boundary, each showing why the agent refunded, declined, or escalated instead of the alternative.

D. Require the agent to state a confidence score with each decision and refund automatically when the score is above 80 percent.

Question 13

Select exactly TWO answers (choose 2).

The refund prompt says 'Be conservative and only approve a refund when you are highly confident it is eligible.' Reviewers see inconsistent decisions on identical situations, especially whether damage 'in transit' qualifies. Select the TWO changes that most directly improve consistency.

A. Replace the general instruction with explicit categorical criteria that name which situations are eligible, ineligible, and must be escalated.

B. Add concrete examples of each outcome, including borderline damage cases, showing how the criteria apply.

C. Tell the agent to think very carefully and double-check each decision before answering.

D. Increase max_tokens so the agent has more room to reason about each refund decision.

E. Require the agent to approve only when it reports a confidence of at least 90 percent in its decision.

Question 14

Select ONE answer.

The agent's case summary panel shows three automatic flags: 'order status verified' (98% accurate), 'billing mismatch detected' (94%), and 'suspected return abuse' (about 45% accurate). Human reviewers now ignore the whole panel, including the accurate flags. Which action best restores trust while the problem is fixed?

A. Keep all three flags and add a disclaimer that some flags may be wrong, so reviewers know to verify them.

B. Keep all three flags but instruct the model to report only high-confidence findings and to skip anything it is unsure about, so that fewer false flags reach the reviewers' panel.

C. Temporarily stop surfacing the 'suspected return abuse' flag, route those cases to a separate review path, and refine its criteria before re-enabling it.

D. Increase the number of flags shown per case so reviewers have more signals to compare against each other.

Question 15

Select exactly THREE answers (choose 3).

A billing dispute session runs for 25 turns. lookup_order returns 40+ fields, the customer raises three issues, and the agent starts forgetting amounts and order numbers stated earlier. Select the THREE changes that best preserve critical information.

A. Spawn a new subagent for every tool call to hold its output so that the main agent's context never contains tool results.

B. Trim lookup_order results to the few fields relevant to the return or dispute (for example with a PostToolUse hook) before they accumulate in context.

C. Keep a persistent 'case facts' block of amounts, dates, order numbers, and statuses in every prompt, outside summarized history.

D. Keep only the last two turns and rely on the model to recall earlier details from its general knowledge.

E. Summarize the whole conversation into two sentences every three turns and discard the earlier messages.

F. Store structured issue data (order ID, amount, status per issue) in a separate context layer so each concern stays distinct.

Question 16

Select ONE answer.

At the start of a chat the customer writes: 'Skip the bot. I want a human agent now.' The agent responds by calling get_customer and lookup_order 'to see if I can quickly fix it' and then offers a solution. The customer replies angrily. What should the agent have done?

A. Continue investigating first, because resolving the issue quickly is the best way to satisfy the customer's underlying goal, and a fast fix will probably make the request for a person unnecessary.

B. Acknowledge the request and escalate to a human right away, including a handoff summary of what is known so far, without first investigating.

C. Run sentiment analysis and escalate only if the customer's frustration score passes a threshold.

D. Ask the customer to explain why they want a human, and escalate only if the reason shows the case is complex.

Question 17

Select ONE answer.

A developer calls the Messages API once per customer turn and sends only the newest user message plus a one-line summary. The agent keeps asking for the order number the customer already gave and repeats the same lookup_order call. The team proposes adding a subagent to hold the conversation memory. What is the better fix?

A. Send the complete conversation history on every request, including prior assistant tool_use blocks and the matching tool_result blocks.

B. Add a memory subagent that stores the conversation and answers the main agent's questions about what was said earlier.

C. Increase the length of the one-line summary so it includes more of what the customer said in earlier turns.

D. Instruct the agent in the system prompt to always ask the customer to repeat details so nothing is assumed.

Scenario 2: Code Generation with Claude Code (Questions 18-31)

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Question 18

Select ONE answer.

You are asked to 'add comprehensive tests' to a 200-file legacy service that has almost no tests and unclear module dependencies. Which approach with Claude Code is most likely to produce useful results?

A. Generate tests for every file in alphabetical order using the same prompt, so coverage grows predictably, and review the results in batches of twenty files as they finish.

B. Map the structure first, identify high-impact and high-risk areas, write a prioritized plan, and adapt the plan as dependencies are discovered.

C. Ask for one test suite for the whole service in a single prompt and review the result at the end.

D. Test only the public API listed in the README, since internal modules are covered indirectly.

Question 19

Select ONE answer.

Claude Code attempts an Edit on a configuration file that contains twelve identical, auto-generated blocks. The edit fails because the target text matches more than once, and no nearby lines are unique within the file. Claude needs to change only the third block. What is the most reliable next step?

A. Run a Bash sed command with a global substitution so every identical block is updated in one pass.

B. Read the full file, produce the modified contents with only the third block changed, and write the updated file with Write.

C. Use Grep to locate the block and Write only that fragment to the file path.

D. Retry the same Edit call several times in case the matching logic picks the right block on a later attempt, since the tool sometimes resolves ties differently between calls.

Question 20

Select exactly TWO answers (choose 2).

A data migration script written by Claude Code mishandles null values and mixed date formats. You have explained the transformation in prose three times and each attempt is interpreted differently. Select the TWO approaches most likely to fix this.

A. Write tests for the expected behavior and edge cases first, then run them and share the failures with Claude to iterate.

B. Repeat the same prose instructions in capital letters so the model gives them more weight.

C. Provide two or three concrete input and expected output examples, including a null value and each date format.

D. Ask Claude to rate its own confidence in the script and ship it if the rating is high.

E. Ask Claude to rewrite the whole script from scratch each time until the output looks right.

Question 21

Select ONE answer.

Your team wants a /map-dependencies command that scans a large repository and produces thousands of lines of analysis. Developers complain that after running it, the main conversation is full of discovery noise and later answers degrade. How should you build it?

A. Create an ordinary command in .claude/commands/ that runs the scan and prints the full output into the main conversation.

B. Create the command in each developer's ~/.claude/commands/ directory so everyone has the same version locally.

C. Add the dependency-mapping instructions to the project CLAUDE.md so they are loaded in every session.

D. Create a skill in .claude/skills/ with a SKILL.md that sets context: fork so the analysis runs in an isolated sub-agent context and returns only its result.

Question 22

Select exactly TWO answers (choose 2).

Your team has five upcoming tasks. Select the TWO that most clearly justify starting in plan mode.

A. Choose between an in-process cache and a shared Redis cache, which have different infrastructure requirements, and implement the choice.

B. Migrate the HTTP client library used across about 45 files to a new library with a different API.

C. Fix a typo in a single user-facing error message.

D. Add a date validation conditional to one existing function whose behavior is fully specified.

E. Add a null check to one function where a stack trace pinpoints the failing line.

Question 23

Select ONE answer.

A new developer joins and Claude Code generates code that ignores the team's naming and testing conventions, while a senior developer's sessions follow them perfectly. The conventions exist only in the senior developer's ~/.claude/CLAUDE.md. How should you resolve this?

A. Ask the new developer to copy the senior developer's user-level file into their own home directory and keep it updated by hand whenever the team's rules change.

B. Move the conventions into the project's CLAUDE.md (or .claude/CLAUDE.md), commit it, and use /memory to confirm the files are loaded.

C. Put the conventions in the new developer's personal settings so Claude applies them only for that person.

D. Add the conventions to .claude/commands/ and ask developers to invoke the command at the start of every session.

Question 24

Select exactly TWO answers (choose 2).

Your repository has Terraform under terraform/ with strict conventions that should only matter there, and test files named *.test.* spread across dozens of directories that must all follow one testing style. You want rules to load automatically and only when relevant. Select the TWO configurations to use.

A. Put every convention into the root CLAUDE.md under headings and rely on Claude to apply the right section.

B. Create .claude/rules/terraform.md with YAML frontmatter paths: ["terraform/**/*"] containing the Terraform conventions.

C. Create .claude/rules/testing.md with YAML frontmatter paths: ["**/*.test.*"] containing the testing conventions.

D. Place a CLAUDE.md in every directory that contains test files with the testing conventions copied into each.

E. Create skills for Terraform and testing and ask developers to invoke the right one before editing.

Question 25

Select ONE answer.

A developer likes the team's project-scoped /review skill but wants a stricter personal variant for their own use, without changing what teammates get. What is the best approach?

A. Edit the project's SKILL.md and commit it so the stricter version becomes the team default.

B. Create a skill in ~/.claude/skills/ with a different name containing the stricter variant.

C. Add the stricter rules to the project CLAUDE.md so Claude applies them whenever the skill runs.

D. Edit the project's SKILL.md locally and just remember not to commit the change.

Question 26

Select ONE answer.

You are writing a /scaffold-endpoint skill. It should be able to create new files but must never run shell commands or delete anything, and if a developer invokes it without an argument it should prompt for the endpoint name. Which SKILL.md configuration best meets this?

A. A rule in the project CLAUDE.md forbidding Bash and deletes in all sessions.

B. Frontmatter that sets allowed-tools to Read and Write only, and an argument-hint such as [endpoint-name] showing the expected argument.

C. A line in the skill body saying 'do not use Bash or delete files' and a note to ask the user for the endpoint name when none is given, so the behavior is documented where the skill is defined.

D. A user-scoped command in ~/.claude/commands/ that wraps the skill and lists the allowed behavior in its description.

Question 27

Select ONE answer.

Claude Code generates function docstrings in different layouts from file to file, even though the prompt includes a paragraph describing the preferred layout. What is the most effective technique to make the output consistent?

A. Write a longer paragraph describing the docstring format with more rules, exceptions, and a note on tone, and place it at the top of the prompt.

B. Include two to four examples of correctly formatted docstrings in the prompt or project instructions.

C. Increase the sampling temperature so the model explores more formats and picks the best.

D. Generate the docstrings as usual and fix the format afterward with a regular expression script.

Question 28

Select ONE answer.

You gave Claude Code three few-shot examples for writing unit tests: a pure function, a function with a database call, and a function with an HTTP call. For a new async generator it produces tests that do not exercise the yield behavior. A teammate suggests listing every function shape with its own rules. Which approach best helps the model generalize?

A. Replace the set with a few diverse examples that include brief reasoning about what is tested and how boundaries are mocked.

B. Create a dedicated subagent per function shape, each with its own examples, and route by detected shape.

C. Provide twenty examples of the database-call case so the model has a very strong template.

D. Write a detailed rule for each function shape, covering what to assert and what to mock, and extend the list whenever a new shape appears in the codebase.

Question 29

Select ONE answer.

After three hours exploring a large Java service in one Claude Code session, answers become inconsistent. Claude now says 'typically, the service layer would handle this' instead of naming the actual classes it found earlier, and it contradicts earlier findings. Which approach best restores reliability?

A. Run /compact repeatedly and trust the generated summaries to keep all class names and call paths.

B. Switch to a model with a larger context window and continue the same session without notes.

C. Record key findings (class names, call paths) in a scratchpad file and consult it for later questions, delegate verbose exploration to subagents, and use /compact when context fills.

D. Add a sentence to the prompt telling Claude to be specific and not to rely on typical patterns.

Question 30

Select ONE answer.

A multi-hour Claude Code migration of 300 files across several agents is interrupted when the machine restarts. Some files were already migrated. You want to continue without redoing or double-applying work. What design is most reliable?

A. Restart the entire migration from the beginning and let the tools overwrite any previously migrated files, on the assumption that re-running the transformation on a migrated file is harmless.

B. Have each agent write its state (completed, in-progress, pending files and decisions) to a known manifest and load it on restart into the agents' prompts.

C. Resume the old session and rely on its stored tool results to know which files were handled.

D. Ask Claude to infer progress by reading the git history and guessing which files look migrated.

Question 31

Select ONE answer.

Claude Code runs the project's full test command through Bash. It prints roughly 6,000 lines, mostly passing tests. Afterward Claude seems to lose track of earlier requirements in the conversation. What is the best way to handle the test run?

A. Clear the conversation after every command so context never grows.

B. Stop running tests during the session and run them only after all code changes are complete.

C. Keep running the full suite and paste all output into the conversation so nothing is missed, since Claude can only judge the suite if it sees every passing test as well as every failure.

D. Run the tests with output limited to failures and a summary (via reporter options), or delegate the run to a subagent, so only relevant results enter the main context.

Scenario 3: Multi-Agent Research System (Questions 32-47)

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question 32

Select ONE answer.

In the research system, the coordinator launches the synthesis subagent with the instruction 'Synthesize the research findings into a draft.' The subagent returns a generic draft with no specific facts and no citations, although the search and analysis subagents produced rich results minutes earlier. What is the most likely fix?

A. Include the complete findings from the search and analysis subagents in the synthesis subagent's prompt, with content separated from source metadata.

B. Rely on the synthesis subagent to read the coordinator's conversation history, which it inherits when the Task tool spawns it, so the coordinator only gives a short instruction about the output format.

C. Have the synthesis subagent repeat the web searches itself so it has its own fresh evidence to summarize.

D. Add a shared memory store that every subagent reads automatically at startup, then tell the coordinator to stop passing results.

Question 33

Select ONE answer.

The web search subagent discovers a PDF URL and, to save time, calls the document analysis subagent directly. In production, a failure in the document analysis step is invisible to the coordinator, and nobody can reconstruct which documents were analyzed. What architecture change is best?

A. Merge the search and document analysis subagents into one agent so no inter-agent calls are required.

B. Keep the direct calls and have each subagent write a log line describing the call to a shared file.

C. Let the document analysis subagent call back the search subagent whenever it needs more context, forming a peer loop.

D. Route all inter-subagent communication through the coordinator, which receives the URL, decides whether to invoke document analysis, and handles any errors.

Question 34

Select ONE answer.

The research coordinator always runs web search, document analysis, synthesis, and report generation for every query. Simple factual questions such as 'In what year was the company founded?' take as long and cost as much as broad research requests. What should change?

A. Have the coordinator analyze each query's complexity and dynamically choose which subagents to invoke, skipping stages that are not needed.

B. Run every stage in parallel for all queries so that total time equals the slowest single stage.

C. Keep the full pipeline but run it on a smaller, faster model for every query to reduce cost.

D. Add a keyword filter before the coordinator that sends any question beginning with 'what' or 'when' straight to a cached answer table that is refreshed nightly from previous research.

Question 35

Select ONE answer.

A developer defines AgentDefinitions for search, analysis, synthesis, and report subagents. When the coordinator runs, it tries to answer research queries by itself and never delegates. Logs show no subagent invocations. The coordinator's allowedTools list contains Read and WebSearch. What should be changed?

A. Add a sentence to the system prompt saying 'You must delegate all research to the subagents and never answer a query directly yourself'.

B. Give each subagent the full set of tools so they can be reached without a coordinator.

C. Rename the subagents so their names better describe their roles, then re-run.

D. Add "Task" (the subagent-spawning tool) to the coordinator's allowedTools so it can spawn subagents.

Question 36

Select ONE answer.

A search subagent's loop ends whenever the assistant's message includes the text 'FINAL ANSWER'. Sometimes the model forgets the phrase and the loop runs until a safety timeout, and sometimes it writes the phrase while still requesting a tool. What is the correct termination logic?

A. End the loop after three iterations in all cases so it cannot run for long.

B. End the loop when the response's stop_reason is 'end_turn', and keep executing requested tools while it is 'tool_use'.

C. Keep the phrase check but add a second and third phrase such as 'DONE' and 'COMPLETE', so that any of them ends the loop, and remind the model in the prompt to use one.

D. End the loop as soon as the assistant message contains any text content.

Question 37

Select ONE answer.

A researcher uses a named session to investigate a market over several days with --resume. Overnight, two of the source documents the agent analyzed were revised. The researcher wants the investigation to continue accurately without re-reading everything. What is the best approach?

A. Resume the session without mentioning the change, since the agent will notice changes in its tools automatically.

B. Delete the session and re-run the whole investigation from the beginning to be safe.

C. Resume the named session and tell the agent which two documents changed, so it re-analyzes those specific files and updates the affected conclusions.

D. Fork the session and let the forked branch assume the original conclusions remain correct.

Question 38

Select ONE answer.

The coordinator currently spawns the web search subagent, waits, then spawns the document analysis subagent in the next turn, even though the two tasks are independent. Its prompt also lists numbered steps ('1. search for X, 2. open each result...') that sometimes block the subagents from adapting to unexpected sources. Which redesign addresses both issues?

A. Emit multiple Task calls in one coordinator response for the independent subagents, and rewrite the prompt to state research goals and quality criteria instead of numbered steps.

B. Have the web search subagent spawn the document analysis subagent as soon as it finds a file, and shorten the prompt.

C. Run both subagents from application code in two parallel threads, and keep the numbered steps in the prompt so each subagent's behavior stays predictable and easy to audit across runs.

D. Replace both subagents with one agent that performs search and analysis in sequence from a long list of instructions.

Question 39

Select ONE answer.

The web search subagent's tool wrapper catches network timeouts and returns an empty results list marked successful. In the final report, the synthesis says 'no evidence was found that vendor X had security incidents'. Later it turns out the search never ran. What change best prevents this?

A. Have the coordinator re-run every empty search with broader terms to see if something appears.

B. Keep empty-success responses but add a low-confidence label to the final report in every case.

C. Separate access failures from valid empty results: return structured error context for timeouts, and a zero-match success only when the search actually ran.

D. Treat every empty result as a failure and retry the search three times with exponential backoff before reporting a problem, logging each attempt for later analysis.

Question 40

Select ONE answer.

The document analysis subagent spends many turns calling list and search tools just to find out which internal reports exist before it can analyze anything. The team has a reliable index of all reports with titles, dates, and topics. How can you reduce these exploratory calls?

A. Tell the agent to guess likely report titles and open them directly.

B. Embed the full text of all reports in the system prompt so the agent already has everything.

C. Add another search tool with a similar name and a broader query syntax, so that the agent has more ways to find reports and can pick whichever works.

D. Expose the report catalog as an MCP resource so the agent can see what is available without exploratory tool calls.

Question 41

Select ONE answer.

The coordinator frequently sends PDF analysis requests to analyze_content and web page requests to analyze_document, although both tools have nearly identical descriptions ('Analyzes content and returns findings'). Which fix is most effective?

A. Rename the tools to purpose-specific names (for example extract_web_results and analyze_document_pdf) and rewrite descriptions with inputs, outputs, and boundaries for each.

B. Keep the names and add a system prompt line saying 'use analyze_content for the internet and analyze_document for files'.

C. Delete one of the tools and route all requests through the remaining one, keeping its generic description.

D. Force the model to try analyze_content first for every request and switch to analyze_document only on an error.

Question 42

Select ONE answer.

The research system uses the same agent instance to write a report and then review it. The reviewer keeps approving the report even though a spot check finds an unsupported claim. What is the best improvement?

A. Ask the generating agent to review its own work again in the same session, with explicit instructions to take a more critical tone this time.

B. Use a second, independent Claude instance without the generator's reasoning context to review the report.

C. Increase the length of the report so the reviewer has more material to evaluate.

D. Remove the review step and rely on the quality of the initial generation.

Question 43

Select ONE answer.

A cited research report passes schema validation: every claim has a URL, the URLs resolve, and the formatting is perfect. A manual audit shows 11% of claims are not actually supported by the cited page. Which verification design is most effective?

A. Ask the writing agent to re-check each of its own citations using extended thinking before finalizing, and to remove any claim it is unsure about so unsupported claims drop out.

B. Add a stricter schema rule that every claim must include at least two URLs.

C. Run the generation three times and keep only claims that appear in at least two runs.

D. Run a separate verifier instance that receives only each claim and its cited excerpt, without the generator's reasoning, and returns supported or unsupported.

Question 44

Select ONE answer.

A single-pass review of 12 analyzed source documents produces thorough comments on the first few, shallow comments on the rest, and contradictory judgments about the same statistic in two documents. What restructuring is best?

A. Run three full-length single-pass reviews and accept only the findings that appear in at least two of them, to filter out one-off errors from any single run.

B. Split the batch into two sets of six documents and review each set in a separate session without any later comparison.

C. Run a focused per-document pass for local issues, then a separate integration pass that compares claims across documents.

D. Ask the reviewer to be consistent across all documents and to treat all documents with equal care.

Question 45

Select ONE answer.

The document analysis subagent extracts the methodology of each paper. It returns null for methodology on papers where the method is described in narrative paragraphs rather than a labelled 'Methods' section. Which prompt change is likely most effective?

A. Pre-process every paper to detect the 'Methods' heading and skip papers without one.

B. Instruct the model to search harder and to avoid returning null whenever possible.

C. Add few-shot examples showing correct extraction from documents with different structures, including one where methodology is embedded in narrative text.

D. Mark methodology as required in the schema so the model must always produce a value.

Question 46

Select ONE answer.

In a research pipeline, the summarization step condenses ten findings into one paragraph. The final report states a statistic but nobody can tell which source it came from. What should the pipeline have done?

A. Require structured claim-source mappings (claim, source, excerpt) from subagents and make synthesis preserve and merge them.

B. Remove statistics from reports to avoid unattributed numbers.

C. Ask the summarizer to add a general line listing all the sources used by the ten findings at the end of the paragraph, in alphabetical order.

D. Have the report generator search the web again to rediscover where each statistic came from.

Question 47

Select ONE answer.

During a research run, the web search subagent times out on one of five sub-topics after retrying locally. The rest of the pipeline succeeds. Which behavior is best for the final deliverable?

A. Retry the failing search indefinitely until it succeeds before producing any output.

B. Silently omit that sub-topic and present the report as complete.

C. Terminate the entire run and tell the user the research failed, since a report with a missing sub-topic would be incomplete and might mislead decision makers.

D. Return structured error context to the coordinator, which continues with the available findings and annotates the report with the coverage gap.

Scenario 4: Developer Productivity with Claude (Questions 48-60)

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question 48

Select ONE answer.

A developer-productivity team built a coordinator with four subagents (explorer, planner, coder, tester) and runs all four for every request. Renaming one local variable in one file now takes minutes, and details discovered by the explorer are often lost in handoffs. What should the team do?

A. Add a fifth subagent that summarizes each handoff so details are not lost between the others.

B. Remove the coordinator and let the four subagents message each other directly as needed.

C. Keep the four-agent pipeline but run the planner and the coder in parallel and cache the explorer's output, then reconcile their results at the end to reduce elapsed time.

D. Let the coordinator assess task scope and handle small, well-scoped changes with a single agent, invoking subagents only when the task needs them.

Question 49

Select ONE answer.

Your exploration subagent has Read, Grep, Glob, Write, and Bash. Twice it modified files while 'just checking something' and sometimes needs commit history to understand why code exists. You want exploration to be strictly read-only but still able to see history. What is the best configuration?

A. Keep Write but add a hook that logs every write so mistakes can be reverted later.

B. Keep Bash and Write and add the instruction 'do not modify files, only run read-only commands'.

C. Restrict it to Read, Grep, and Glob and add a narrowly scoped git_history tool that only returns read-only log output for a given path.

D. Remove all tools and have the coordinator paste file contents and history into the subagent's prompt.

Question 50

Select ONE answer.

You need every caller of fetchInvoice. The function is defined in billing/api.ts, re-exported under the alias getInvoice by a barrel file, and wrapped by billingClient.fetch(). A Grep for 'fetchInvoice' returns 4 files, but runtime traces show more callers. What is the best approach?

A. Grep only for 'fetchInvoice' with a case-insensitive flag, since the alias is probably a variation of the same word.

B. Use Glob with the pattern **/*invoice* to list candidate files and assume they contain all callers.

C. Read every file in the repository into context and search by eye for any call that looks like an invoice fetch, working through the directories one at a time.

D. First identify all exported names and wrappers of the function (the alias, re-export, and wrapper), then Grep for each name across the codebase.

Question 51

Select ONE answer.

Engineers asked the agent for open bugs linked to a failing service. It keeps using built-in Grep on exported issue text files rather than the team's MCP tool search_issues, which supports JQL filtering and linked-issue traversal. The tool's description reads: 'Searches issues.' What is the best fix?

A. Remove Grep from the agent's allowed tools so that it has to use the MCP tool for every search it performs, whether the target is code, logs, or issues.

B. Replace the MCP tool with a custom wrapper script that calls the same API and is run through Bash.

C. Rewrite the MCP tool description to explain its capabilities, inputs such as JQL, outputs, and when to prefer it over text search.

D. Add 'always use search_issues' to CLAUDE.md for every task the agent performs.

Question 52

Select ONE answer.

An engineer must decide how to change a legacy billing module whose behavior is poorly documented. Exploring it will involve reading dozens of files, and the engineer wants to avoid filling the main conversation with raw file contents. Which approach fits best?

A. Read all of the module's files into the main conversation first so that every detail is available while designing the change, and then begin editing.

B. Start editing right away and learn the structure as the changes break things.

C. Use plan mode and delegate discovery to the Explore subagent, which returns a summary so the main conversation keeps room for design.

D. Create a separate custom subagent for each source file in the module and ask the main agent to merge their outputs.

Question 53

Select ONE answer.

A monorepo has packages with different standards: the api package needs API and testing standards, the data package needs only data and testing standards. A single 2,000-line root CLAUDE.md makes sessions noisy and developers keep editing conflicting parts. What is a more maintainable approach?

A. Copy the relevant standards text into each package's CLAUDE.md so every package is self-contained.

B. Keep standards in separate files and use @import in each package's CLAUDE.md to include only the standards relevant to that package.

C. Convert each standard into a skill and require developers to invoke the right skills for their package.

D. Keep the single root CLAUDE.md but add headings for each package and rely on Claude to ignore irrelevant sections.

Question 54

Select ONE answer.

A developer unfamiliar with caching asks Claude Code to 'add a cache to the product lookup service'. Claude implements a simple time-based cache. Later problems appear: stale prices after updates and a thundering herd on cold start. What technique should the developer have used before implementation?

A. Ask Claude to implement the simplest version first and review any production incidents afterward, then iterate on the design as problems are found.

B. Ask Claude to interview them first about cache invalidation, failure modes, and consistency needs before writing code.

C. Request three completely different caching implementations and pick the one that looks cleanest.

D. Tell Claude to use a well-known caching library and avoid writing custom logic.

Question 55

Select ONE answer.

A team has two pieces of guidance. One is 'always use strict TypeScript settings and never use any'. The other is a 40-step release checklist used a few times each month. Where should each be placed in Claude Code?

A. Put the TypeScript rule in CLAUDE.md so it always applies, and make the release checklist a skill invoked when releasing.

B. Make both skills and rely on developers to invoke them as needed.

C. Put the checklist in CLAUDE.md and the TypeScript rule in a skill so the longer content is always visible.

D. Put both in CLAUDE.md so everything is available in every session without anyone needing to remember to invoke a command or skill or to search for it.

Question 56

Select ONE answer.

You must migrate a logging library across 47 files, with some modules using custom wrappers. The team wants an agreed approach before edits begin and then wants the work completed efficiently. Which workflow is best?

A. Remain in plan mode through implementation so that each edit is first proposed for approval.

B. Write a long manual migration guide and have developers perform the edits by hand.

C. Start with direct execution and fix modules one at a time, adjusting the approach as wrappers and dependencies appear, which keeps each individual change small.

D. Use plan mode to investigate usage and design the migration approach for review, then switch to direct execution to implement the approved plan.

Question 57

Select exactly TWO answers (choose 2).

The developer assistant has Grep, Glob, Read, and an MCP tool find_symbol. For requests like 'where is the E_TIMEOUT error raised?' it sometimes picks Glob, and for 'find all *.spec.ts files' it sometimes picks Grep. Select the TWO changes that best improve tool selection on ambiguous requests.

A. Add ten more overlapping search tools so the model can choose the closest match.

B. Add a few targeted few-shot examples showing ambiguous requests and why one tool was chosen over the plausible alternative.

C. Set tool_choice to 'any' so the model has to choose a tool on each turn.

D. Expand each tool description with input formats, example queries, and boundaries stating when to use it versus similar tools.

E. Add a keyword router in application code that always selects the tool before the model sees the request.

Question 58

Select ONE answer.

When summarizing legacy modules, the agent flags every use of global state as a defect, even in well-contained, intentional configuration singletons. Developers start ignoring the summaries. Which prompt change best reduces these false flags while still catching real problems?

A. Add examples that distinguish acceptable patterns (contained, intentional singletons) from genuine issues (mutable state shared across threads), with reasoning.

B. Ask the agent to list every global variable without judgment and let developers decide.

C. Stop flagging global state entirely in all future summaries.

D. Add the instruction 'Only flag global state when you are very sure it is a real problem and skip anything borderline' to the summary prompt, then review the false flags weekly.

Question 59

Select ONE answer.

Asked to 'explain this legacy module well', the agent returns varied answers: sometimes a function-by-function tour, sometimes a high-level essay. Engineers want a consistent, useful orientation. Which prompt change is best?

A. State explicit criteria for the explanation: entry points, side effects, external dependencies, data it mutates, and known risky areas.

B. Request the explanation three times and choose the best by eye.

C. Tell the agent to match the style of the last answer it gave to the same engineer.

D. Ask for the explanation to be 'thorough but concise' and 'appropriate to the module's importance', so that the agent aims for a sensible middle ground in every answer.

Question 60

Select ONE answer.

A refactoring run is split among four worker agents with a coordinator. The coordinator crashes after two workers have written their changes. After restart, the coordinator re-delegates all four work packages and two files end up with duplicated edits. What design prevents this?

A. Restart all four workers from scratch and instruct each one to check carefully for existing edits and not to duplicate any change that was already made.

B. Have the coordinator resume its old session and trust its recollection of which workers finished.

C. Each worker exports its completed units to a known manifest; on resume the coordinator loads it and delegates only the unfinished units.

D. Ask a fifth agent to scan the repository afterward and remove anything that looks duplicated.

End of examination. Stop here, then check your answers using the key that follows.

5. Quick answer key and score tracker

Mark each question you answered correctly (all required selections, no wrong ones). Then complete the domain tracker below.

No.	Ans.	Domain	Scen.	Mine
1	**A**	D1	S1	
2	**A**	D1	S1	
3	**B**	D1	S1	
4	**D**	D1	S1	
5	**D**	D1	S1	
6	**A**	D1	S1	
7	**C**	D1	S1	
8	**A**	D2	S1	
9	**C**	D2	S1	
10	**D**	D2	S1	
11	**B**	D2	S1	
12	**C**	D4	S1	
13	**A,B**	D4	S1	
14	**C**	D4	S1	
15	**B,C,F**	D5	S1	
16	**B**	D5	S1	
17	**A**	D5	S1	
18	**B**	D1	S2	
19	**B**	D2	S2	
20	**A,C**	D3	S2	
21	**D**	D3	S2	
22	**A,B**	D3	S2	
23	**B**	D3	S2	
24	**B,C**	D3	S2	
25	**B**	D3	S2	
26	**B**	D3	S2	
27	**B**	D4	S2	
28	**A**	D4	S2	
29	**C**	D5	S2	
30	**B**	D5	S2	

No.	Ans.	Domain	Scen.	Mine
31	**D**	D5	S2	
32	**A**	D1	S3	
33	**D**	D1	S3	
34	**A**	D1	S3	
35	**D**	D1	S3	
36	**B**	D1	S3	
37	**C**	D1	S3	
38	**A**	D1	S3	
39	**C**	D2	S3	
40	**D**	D2	S3	
41	**A**	D2	S3	
42	**B**	D4	S3	
43	**D**	D4	S3	
44	**C**	D4	S3	
45	**C**	D4	S3	
46	**A**	D5	S3	
47	**D**	D5	S3	
48	**D**	D1	S4	
49	**C**	D2	S4	
50	**D**	D2	S4	
51	**C**	D2	S4	
52	**C**	D3	S4	
53	**B**	D3	S4	
54	**B**	D3	S4	
55	**A**	D3	S4	
56	**D**	D3	S4	
57	**B,D**	D4	S4	
58	**A**	D4	S4	
59	**A**	D4	S4	
60	**C**	D5	S4	

Domain-level score tracker

Domain	Questions	Number list	My correct	% correct
1. Agentic Architecture & Orchestration	16	1, 2, 3, 4, 5, 6, 7, 18, 32, 33, 34, 35, 36, 37, 38, 48		
2. Tool Design & MCP Integration	11	8, 9, 10, 11, 19, 39, 40, 41, 49, 50, 51		
3. Claude Code Configuration & Workflows	12	20, 21, 22, 23, 24, 25, 26, 52, 53, 54, 55, 56		
4. Prompt Engineering & Structured Output	12	12, 13, 14, 27, 28, 42, 43, 44, 45, 57, 58, 59		
5. Context Management & Reliability	9	15, 16, 17, 29, 30, 31, 46, 47, 60		
Total	**60**			

Interpretation: percent-correct here is a practice signal only. The real exam's pass/fail decision is based on a single scaled score (cut score 720 of 1,000); domain percentages on the real score report are informational.

6. Detailed answer key and explanations

Each entry gives the correct answer, why it is correct, why every other option is wrong, the concept tested, the detail in the scenario that decides the answer, and a production lesson.

Q1 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.7 | Intermediate | Single answer

Correct answer: A - Start a new session seeded with a structured summary of confirmed case facts, and call lookup_order again for current order state.

Why the correct answer is correct: When most of what a session remembers is tool output that can change (order status, shipment state), resuming it carries stale facts forward. The guide's guidance is that starting a new session with a structured summary is more reliable than resuming with stale tool results. The summary keeps durable facts (customer ID, order number, what the customer asked for) and the agent re-queries volatile state before answering.

Option-by-option:

A. **[CORRECT]** Start a new session seeded with a structured summary of confirmed case facts, and call lookup_order again for current order state.
Correct. Prior tool results are stale, so a fresh session with an injected summary of durable facts plus a live lookup gives the agent accurate state.

B. **[INCORRECT]** Fork the session with fork_session and ask the forked branch to answer from the saved analysis without further tool calls.
A fork starts from the same history, including the stale result. Forking is for exploring divergent approaches, not for refreshing outdated data.

C. **[INCORRECT]** Switch to a model with a larger context window so that the newest user message outweighs older tool output in the history.
Context size does not change which facts are true. Older tool results do not become less influential simply because the window is larger.

D. **[INCORRECT]** Resume the same session and add a system prompt line saying that earlier tool results may be outdated and should be treated cautiously.
The stale 'processing' result remains in context and the model must guess which earlier results to distrust. A caution line is probabilistic and does not refresh the data.

Key concept: Session resumption versus fresh start with injected summary (Task 1.7).
Exam reasoning: The elapsed two days and the changed warehouse status are the decisive details: the problem is stale tool results, not model capability or prompt wording. Only an option that refreshes the data fixes the root cause.
Production lesson: Treat tool results as a cache with a lifetime. On resume, carry forward durable case facts and re-fetch anything that can change.

Q2 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Intermediate | Single answer

Correct answer: A - In a PreToolUse hook for process_refund, load the order and reject the call unless its customer ID equals the verified customer ID for the session.

Why the correct answer is correct: Authentication (who is this person) and authorization (are they allowed to act on this resource) are different checks. The existing gate only proves that authentication happened. A tool-call interception hook for process_refund, or equivalent server-side validation, must compare the order's owner to the verified customer and block mismatches. Doing so deterministically avoids depending on the model's compliance.

Option-by-option:

A. **[CORRECT]** In a PreToolUse hook for process_refund, load the order and reject the call unless its customer ID equals the verified customer ID for the session.
Correct. This enforces authorization (the verified person owns this order) deterministically, at the point the side effect would occur.

B. **[INCORRECT]** Ask the caller to restate the email address on the account before each refund and compare it to the get_customer record in the prompt.
Restating an email is another identity check (authentication). The failure here is that a verified identity acted on an order it does not own.

C. **[INCORRECT]** Update the system prompt to tell the agent that it must confirm an order belongs to the verified customer before issuing a refund.
This relies on the model remembering to perform the check every time. Financial side effects need a guarantee, and prompts have a non-zero failure rate.

D. **[INCORRECT]** Extend the existing prerequisite so it also requires that lookup_order has been called earlier in the session for the same order number before process_refund is allowed to run.
Calling lookup_order only proves the order exists. It does not prove that the verified customer owns it, so the unauthorized refund still passes.

Key concept: Programmatic enforcement of authorization before a financial side effect (Task 1.4, 1.5).
Exam reasoning: The audit shows a verified customer ID and a refund on someone else's order. That is an ownership mismatch, so the correct fix must compare the order's customer to the verified customer, not merely add another identity step or another reminder.
Production lesson: Gate every mutating tool on resource-level ownership checks, enforced in code or in the tool backend. Do not stop at 'the user was verified'.

Q3 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.1 | Challenging | Single answer

Correct answer: B - Continue while stop_reason is 'tool_use', run the tools and append their results, finish on 'end_turn', and keep an iteration limit only as an escalating backstop.

Why the correct answer is correct: The loop should be driven by the API's stop_reason: when it is 'tool_use', run the requested tools, append the tool_result blocks to the conversation, and call the model again; when it is 'end_turn', return the final response. Assistant text can accompany tool_use, so checking for text ends the loop too early. A hard cap used as the main terminator cuts off legitimate multi-step cases; it belongs only as a safety backstop, ideally escalating to a human when reached.

Option-by-option:

A. **[INCORRECT]** Keep the text check, but instruct the model never to write any text before calling a tool, so that the first text block it produces always signals the final answer to the customer.
This still treats assistant text as a completion indicator and depends on the model obeying a prompt rule. Text can legitimately appear alongside tool_use blocks.

B. **[CORRECT]** Continue while stop_reason is 'tool_use', run the tools and append their results, finish on 'end_turn', and keep an iteration limit only as an escalating backstop.
Correct. The stop_reason is the authoritative signal. A backstop that escalates handles runaway loops without being the normal way the loop ends.

C. **[INCORRECT]** Replace the loop with a fixed sequence of get_customer, lookup_order, then process_refund so each request runs the same three steps.
A fixed sequence removes model-driven decisions, so account issues and disputes that need other steps cannot be handled, and it still has no correct termination logic.

D. **[INCORRECT]** Raise the iteration cap to 25 and also stop the loop when the response includes a phrase such as 'I have completed your request'.
Parsing natural-language phrases for termination is brittle, and a larger cap is still an arbitrary number acting as the primary stopping rule.

Key concept: Agentic loop control flow and anti-patterns (Task 1.1).
Exam reasoning: Two symptoms appear together: early exit when text is present, and abrupt exit at five iterations. Both come from using the wrong termination signals (text content and a cap), so the fix must replace both with stop_reason-based control.
Production lesson: Never infer completion from message text or an arbitrary count. Use stop_reason, and make caps a guard that fails safely.

Q4 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.5 | Intermediate | Single answer

Correct answer: D - Add a PostToolUse hook that converts all dates to one format and maps numeric status codes to labeled values before the model sees any tool result.

Why the correct answer is correct: PostToolUse hooks intercept tool results before the model processes them, which makes them the right place to normalize timestamps and status codes. The model then compares dates in one format and reads labeled statuses, and the transformation is guaranteed rather than requested.

Option-by-option:

A. **[INCORRECT]** Describe the conversion formula for Unix timestamps and the meaning of each numeric status code in the system prompt, and ask the agent to convert every value before it reasons about eligibility.
The model must then perform conversions on every call, and any lapse produces wrong eligibility decisions. It is probabilistic where the problem needs determinism.

B. **[INCORRECT]** Lengthen each tool's description to explain the format of its output fields so the model knows how to interpret each response.
Better descriptions help selection and interpretation, but they still leave the arithmetic and mapping to the model on every call.

C. **[INCORRECT]** Add a convert_timestamp tool and instruct the agent to call it whenever a date appears in any tool result.
This adds extra tool calls, tokens, and more chances to skip a call. The normalization is mechanical and should not depend on model initiative.

D. **[CORRECT]** Add a PostToolUse hook that converts all dates to one format and maps numeric status codes to labeled values before the model sees any tool result.
Correct. A hook normalizes heterogeneous results deterministically, so the model always reasons over consistent data.

Key concept: PostToolUse hooks for data normalization (Task 1.5).
Exam reasoning: The tools return inconsistent formats and the failures are intermittent, which points to model-side conversion errors. A deterministic transformation layer is better than asking the model to convert correctly each time.
Production lesson: Normalize at the integration boundary. Keep the model's input consistent and let code do code's work.

Q5 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.6 | Challenging | Single answer

Correct answer: D - Split the message into three concerns, run read-only investigation for them in parallel from shared verified-customer and order facts, reconcile the findings, then execute any refund once.

Why the correct answer is correct: The guide describes decomposing multi-concern requests into distinct items and investigating them in parallel using shared context before synthesizing a unified resolution. Because the return and dispute interact, the side effect (the refund) must happen once, after the facts are reconciled. Reads can be parallel; writes need coordination.

Option-by-option:

A. **[INCORRECT]** Process the concerns strictly in the order mentioned, starting each from a clean context so earlier issues cannot influence later ones.
This is slower and discards the shared facts that matter. The later dispute depends on what happened with the return.

B. **[INCORRECT]** Launch three isolated subagents that each handle one concern end to end, including calling process_refund if their analysis supports it.
Isolated agents do not share facts. The return and the dispute concern the same charge, so both could issue refunds for it.

C. **[INCORRECT]** Escalate the entire message to a human agent because it contains several issues and one of them involves money.
Multi-concern requests are within the agent's remit. Escalating reduces first-contact resolution when the issues can be resolved with the available tools.

D. **[CORRECT]** Split the message into three concerns, run read-only investigation for them in parallel from shared verified-customer and order facts, reconcile the findings, then execute any refund once.
Correct. It parallelizes the safe, independent lookups, preserves shared context, and makes the one side-effect decision after the interacting facts are combined.

Key concept: Task decomposition with shared context and controlled side effects (Task 1.4, 1.6).
Exam reasoning: The sentence 'the return refund affects the amount in dispute' shows the concerns interact. That rules out fully independent agents that can each refund, and it rules out an unnecessary escalation.
Production lesson: Parallelize reads, serialize writes. Reconcile interacting facts before any irreversible action.

Q6 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Foundational | Single answer

Correct answer: A - A structured handoff summary with customer ID, root cause, refund amount involved, and the recommended next action.

Why the correct answer is correct: Humans who lack transcript access depend on the handoff summary. The guide specifies compiling structured summaries (customer ID, root cause, refund amount, recommended action) when escalating mid-process so the human can resolve the case without re-investigating.

Option-by-option:

A. **[CORRECT]** A structured handoff summary with customer ID, root cause, refund amount involved, and the recommended next action.

Correct. The human can act immediately without the transcript.

B. [INCORRECT] Only the customer's name, with an instruction for the human agent to find the case in the CRM by searching that name.
Names are often ambiguous (multiple matches), and it still omits the cause, amounts, and recommendation.

C. [INCORRECT] A message asking the customer to describe the problem again to the human agent when the connection is made.
That recreates the exact problem: the customer repeats themselves and context is lost.

D. [INCORRECT] The full raw conversation text and every tool output pasted into the escalation note as one large block, so the human agent has complete information available to search through.
This buries the key facts in noise and includes sensitive or irrelevant data. A focused summary is easier to act on.

Key concept: Structured handoff protocols (Task 1.4).
Exam reasoning: The stated constraint that humans cannot see the transcript means the escalation payload must carry all necessary facts itself, in a usable structure.
Production lesson: Design handoffs as if the receiver has no other context, because often they do not.

Q7 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.7 | Intermediate | Single answer

Correct answer: C - Use fork_session for two branches from the shared investigation; each drafts a resolution without executing refunds, and only the chosen resolution is executed, once.

Why the correct answer is correct: fork_session creates independent branches from a shared baseline so divergent approaches can be explored without contaminating each other. Because process_refund is a side effect, exploration should be draft-only, with execution happening once after a decision.

Option-by-option:

A. [INCORRECT] Run strategy one in the current session and then instruct the agent to disregard it and try strategy two in the same session.
The first strategy remains in context and biases the second, and any executed refund stays executed.

B. [INCORRECT] Fork the session and let both branches execute their own resolution against the customer's order, then issue a compensating adjustment to reverse whichever one the lead does not choose.
Executing both creates real side effects and customer-visible actions that may not be cleanly reversible.

C. [CORRECT] Use fork_session for two branches from the shared investigation; each drafts a resolution without executing refunds, and only the chosen resolution is executed, once.
Correct. Forks share the baseline, and keeping branches draft-only prevents duplicate or conflicting refunds.

D. [INCORRECT] Resume the same named session in two terminals with --resume and let each terminal pursue a different resolution.
Both processes continue the same conversation lineage rather than creating independent branches, and each may execute actions.

Key concept: fork_session for divergent exploration; side-effect safety (Task 1.7).
Exam reasoning: The prototype's ability to refund in both branches is the hazard. The best answer keeps the benefit of forking while removing the duplicate-action risk.
Production lesson: Exploration branches should be read-only or draft-only. Commit side effects once.

Q8 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.1 | Intermediate | Single answer

Correct answer: A - Remove the keyword-triggered routing rule from the system prompt and rely on the tool descriptions, which already state each tool's inputs and boundaries.

Why the correct answer is correct: Tool descriptions are the main mechanism the model uses to choose tools, but system prompt wording can override them. Keyword-sensitive instructions (here 'charge' maps to lookup_invoice) create unintended tool associations. The fix is to remove or rewrite the keyword rule rather than pile on more.

Option-by-option:

A. [CORRECT] Remove the keyword-triggered routing rule from the system prompt and rely on the tool descriptions, which already state each tool's inputs and boundaries.
Correct. The keyword rule overrides well-written descriptions and creates an unintended association between 'charge' and one tool.

B. [INCORRECT] Merge lookup_invoice and lookup_order into a single lookup_record tool so there is no routing decision to make.
Merging hides two different backends behind one interface and discards the clear descriptions that were working. The

root cause is the prompt rule.

C. **[INCORRECT]** Set tool_choice to force lookup_invoice on every turn so the behavior at least becomes predictable for billing messages.
Forcing one tool on every turn prevents the agent from ever using lookup_order or answering, and it treats a prompt wording problem as a configuration problem.

D. **[INCORRECT]** Add more keywords such as 'shipped', 'delivery', and 'tracking' to the prompt rule and map them to lookup_order, so that delivery questions are routed to the right tool.
This extends a brittle keyword scheme. Mixed messages such as the charge-for-undelivered-order case will still match whichever rule the model weighs most.

Key concept: Keyword-sensitive system prompt instructions overriding tool descriptions (Task 2.1).
Exam reasoning: The detail that tool descriptions are already good and the problem began after a prompt update points to the prompt rule as the cause.
Production lesson: After any tool-selection regression, diff the system prompt as well as the tool descriptions.

Q9 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.4 | Intermediate | Single answer

Correct answer: C - Put the shared servers in the project's .mcp.json using ${CRM_API_TOKEN} environment variable expansion, and add the experimental server to the engineer's user-scoped ~/.claude.json.

Why the correct answer is correct: MCP server scope matters: project-level .mcp.json is for shared team tooling and supports environment variable expansion such as ${GITHUB_TOKEN}-style references so credentials stay out of version control. User-level ~/.claude.json holds personal or experimental servers. All configured servers' tools are available simultaneously.

Option-by-option:

A. **[INCORRECT]** Commit .mcp.json with the literal token because the repository is private, and tell the experimenting engineer to edit the same file locally.
A private repository still exposes the secret to everyone with access and its history, and local edits to a shared file risk accidental commits.

B. **[INCORRECT]** Write the shared server definitions and the literal token in CLAUDE.md so Claude Code reads them at the start of each session.
CLAUDE.md holds instructions and context, not MCP server configuration, and it would commit a secret in plain text.

C. **[CORRECT]** Put the shared servers in the project's .mcp.json using ${CRM_API_TOKEN} environment variable expansion, and add the experimental server to the engineer's user-scoped ~/.claude.json.
Correct. Project scope is shared through version control, expansion keeps secrets out of the file, and user scope keeps the experiment personal.

D. **[INCORRECT]** Ask each engineer to add all servers, including the shared ones, to their own ~/.claude.json using a token from the team password manager.
This is not shared via version control, so engineers do not get the servers when they clone; setup must be replicated by hand and configurations drift.

Key concept: MCP server scoping and credential management (Task 2.4).
Exam reasoning: Two requirements pull in different directions: share with the team on clone, and keep one server personal. They map directly to project scope with expansion and user scope.
Production lesson: Share the configuration, never the secret. Use environment variables for credentials.

Q10 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.3 | Intermediate | Single answer

Correct answer: D - Give the resolution agent only the four support tools; keep administrative capabilities out of its tool set and reachable only through escalate_to_human and staff tooling.

Why the correct answer is correct: Giving an agent far more tools than it needs degrades selection reliability and increases misuse risk. The guide describes scoping each agent to the tools it needs, with limited cross-role tools for high-frequency needs. Administrative actions that require human approval belong behind the escalation path, not in the resolution agent's tool list.

Option-by-option:

A. **[INCORRECT]** Set tool_choice to 'any' so the model must call one of the 18 tools on every turn, which reduces conversational mistakes.
tool_choice 'any' only guarantees that some tool is called. It does nothing to prevent the wrong tool, and it forces calls on turns that need none.

B. **[INCORRECT]** Add a second model that reviews each tool call and vetoes administrative calls, while keeping the full tool list for the main agent.
This adds latency and cost and relies on another probabilistic check, when removing unnecessary tools solves the problem directly.

C. **[INCORRECT]** Keep all 18 tools and add a prompt paragraph explaining which tools are reserved for staff, with a firm warning that the agent must never call them even when a customer asks directly.
Large tool sets raise selection difficulty, and prompt warnings leave destructive tools callable. Compliance is probabilistic.

D. **[CORRECT]** Give the resolution agent only the four support tools; keep administrative capabilities out of its tool set and reachable only through escalate_to_human and staff tooling.
Correct. A small, role-specific tool set improves selection and applies least privilege.

Key concept: Tool distribution, scoped access, and least privilege (Task 2.3).
Exam reasoning: The stem says administrative tools need staff approval anyway, so the agent has no legitimate use for them. Removing them is both simpler and safer than controlling their use.
Production lesson: The safest tool is the one the agent cannot call. Scope the tool set to the role.

Q11 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.2 | Intermediate | Single answer

Correct answer: B - Return an MCP error result (isError) with structured metadata: an error category, an isRetryable flag, and a customer-appropriate description.

Why the correct answer is correct: The isError pattern lets MCP tools signal failure to the agent. Distinguishing transient, validation, business, and permission errors, and indicating whether each is retryable, enables the right recovery: retry timeouts, fix input errors, and explain or escalate policy failures with a clear customer-friendly message.

Option-by-option:

A. **[INCORRECT]** Return a verbose stack trace and server log excerpt so the model can work out the cause by reading it.
Stack traces are noisy and may leak internals. The agent needs a small, stable error contract it can act on.

B. **[CORRECT]** Return an MCP error result (isError) with structured metadata: an error category, an isRetryable flag, and a customer-appropriate description.
Correct. The agent can retry transient failures, correct validation errors, and explain or escalate business-rule failures.

C. **[INCORRECT]** Return an empty result for all failures so that the agent treats the refund as pending and tries again later in the conversation.
An empty successful result hides the failure and risks telling the customer a refund is coming when it is not.

D. **[INCORRECT]** Keep the generic message but add a system prompt rule telling the agent to retry any failed refund up to three times with a short delay, and to escalate if it still fails.
Retrying a non-existent order or a policy violation cannot succeed, and the agent still cannot tell failure causes apart.

Key concept: Structured error responses with category and retryability (Task 2.2).
Exam reasoning: Three different causes produce one message, so the agent applies one recovery. Only richer error metadata lets the agent choose among recoveries.
Production lesson: Design the error contract as an API for the agent: category, retryable, human-readable explanation.

Q12 | Domain 4: Prompt Engineering & Structured Output | Scenario 1: Customer Support Resolution Agent

Task statement 4.2 | Challenging | Single answer, secondary: D5

Correct answer: C - Replace them with a few targeted examples near the decision boundary, each showing why the agent refunded, declined, or escalated instead of the alternative.

Why the correct answer is correct: Few-shot examples are most useful for ambiguous cases when they show the reasoning for choosing one action over plausible alternatives, and they help the model generalize judgment to novel patterns. A set that only shows the typical case teaches the model to pattern-match on surface features.

Option-by-option:

A. **[INCORRECT]** Add twenty more standard damaged-item examples with photo evidence so the agent sees the refund pattern more often and recognizes eligible cases with higher confidence.
More of the same pattern strengthens the overgeneralization. The agent needs to see where the pattern stops applying.

B. **[INCORRECT]** Remove the examples and add the instruction 'when in doubt, approve the refund to keep customers satisfied'.
That removes the escalation judgment entirely and makes the problem of unauthorized exceptions worse.

C. **[CORRECT]** Replace them with a few targeted examples near the decision boundary, each showing why the agent refunded, declined, or escalated instead of the alternative.
Correct. Contrasting examples with reasoning teach the distinction the agent is currently missing and let it generalize.

D. **[INCORRECT]** Require the agent to state a confidence score with each decision and refund automatically when the score is above 80 percent.
Self-reported confidence is poorly calibrated, and the agent is already confidently wrong on these cases.

Key concept: Targeted few-shot examples for ambiguity (Task 4.2, 5.2).
Exam reasoning: The failures are look-alike cases outside policy, which signals missing contrast in the examples rather than a lack of volume.
Production lesson: Choose examples for the edges of the decision boundary, and include why, not just what.

Q13 | Domain 4: Prompt Engineering & Structured Output | Scenario 1: Customer Support Resolution Agent

Task statement 4.1 | Intermediate | Select 2

Correct answer: A, B - Replace the general instruction with explicit categorical criteria that name which situations are eligible, ineligible, and must be escalated. / Add concrete examples of each outcome, including borderline damage cases, showing how the criteria apply.

Why the correct answer is correct: General instructions such as 'be conservative' do not improve precision the way specific categorical criteria do. Defining what to approve, decline, and escalate, with concrete examples for each, gives consistent classification. Both correct choices make the standard explicit.

Option-by-option:

A. **[CORRECT]** Replace the general instruction with explicit categorical criteria that name which situations are eligible, ineligible, and must be escalated.
Correct. Specific categories give the model something concrete to apply and audit.

B. **[CORRECT]** Add concrete examples of each outcome, including borderline damage cases, showing how the criteria apply.
Correct. Examples anchor the criteria and show boundary handling.

C. **[INCORRECT]** Tell the agent to think very carefully and double-check each decision before answering.
This adds no new information about what the policy requires, so inconsistency is likely to persist.

D. **[INCORRECT]** Increase max_tokens so the agent has more room to reason about each refund decision.
Output length limits are not the cause of inconsistent criteria.

E. **[INCORRECT]** Require the agent to approve only when it reports a confidence of at least 90 percent in its decision.
Confidence thresholds are a vague proxy and self-reported confidence is poorly calibrated. They do not define what counts as eligible.

Key concept: Explicit criteria and examples instead of vague conservatism (Task 4.1).
Exam reasoning: The inconsistency on identical situations shows the standard is undefined. Only options that define the standard address that.
Production lesson: Replace adjectives like 'conservative' with testable rules and examples.

Q14 | Domain 4: Prompt Engineering & Structured Output | Scenario 1: Customer Support Resolution Agent

Task statement 4.1 | Intermediate | Single answer

Correct answer: C - Temporarily stop surfacing the 'suspected return abuse' flag, route those cases to a separate review path, and refine its criteria before re-enabling it.

Why the correct answer is correct: High false positive categories undermine trust in accurate categories. The guide recommends temporarily disabling a noisy category to restore trust while improving its prompt with explicit criteria, rather than relying on general confidence-based filtering.

Option-by-option:

A. **[INCORRECT]** Keep all three flags and add a disclaimer that some flags may be wrong, so reviewers know to verify them.
A disclaimer does not repair the credibility loss, and reviewers will keep ignoring the panel.

B. **[INCORRECT]** Keep all three flags but instruct the model to report only high-confidence findings and to skip anything it is unsure about, so that fewer false flags reach the reviewers' panel.
A vague confidence filter does not fix the criteria that cause false positives.

C. **[CORRECT]** Temporarily stop surfacing the 'suspected return abuse' flag, route those cases to a separate review path, and refine its criteria before re-enabling it.
Correct. Removing the noisy category protects trust in the accurate categories while the prompt is improved.

D. **[INCORRECT]** Increase the number of flags shown per case so reviewers have more signals to compare against each other.
More signals add more noise and do not address the 45% false positive category.

Key concept: False positive rates and developer or reviewer trust (Task 4.1).
Exam reasoning: Reviewers ignore the entire panel because of one category, so isolating that category is the fastest way to restore the value of the others.
Production lesson: Ship only categories you can stand behind. Fix or hide the rest.

Q15 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.1 | Intermediate | Select 3

Correct answer: B, C, F - Trim lookup_order results to the few fields relevant to the return or dispute (for example with a PostToolUse hook) before they accumulate in context. / Keep a persistent 'case facts' block of amounts, dates, order numbers, and statuses in every prompt, outside summarized history. / Store structured issue data (order ID, amount, status per issue) in a separate context layer so each concern stays distinct.

Why the correct answer is correct: Three techniques from the guide apply: trim verbose tool outputs to relevant fields, extract transactional facts into a persistent case-facts block outside summarized history, and keep structured per-issue data in a separate layer. Progressive summarization is the very mechanism that loses numbers and dates.

Option-by-option:

A. **[INCORRECT]** Spawn a new subagent for every tool call to hold its output so that the main agent's context never contains tool results.
The main agent needs the relevant results to reason, and adding agents per call adds complexity without solving the loss of key facts.

B. **[CORRECT]** Trim lookup_order results to the few fields relevant to the return or dispute (for example with a PostToolUse hook) before they accumulate in context.
Correct. Verbose tool output consumes tokens disproportionately to its relevance.

C. **[CORRECT]** Keep a persistent 'case facts' block of amounts, dates, order numbers, and statuses in every prompt, outside summarized history.
Correct. Transactional facts are protected from summarization loss.

D. **[INCORRECT]** Keep only the last two turns and rely on the model to recall earlier details from its general knowledge.
The model cannot recall session-specific facts it no longer has in context.

E. **[INCORRECT]** Summarize the whole conversation into two sentences every three turns and discard the earlier messages.
Aggressive progressive summarization turns precise numbers and dates into vague statements.

F. **[CORRECT]** Store structured issue data (order ID, amount, status per issue) in a separate context layer so each concern stays distinct.
Correct. Multi-issue sessions need per-issue structured state.

Key concept: Context preservation in long, multi-issue sessions (Task 5.1).
Exam reasoning: Forgotten amounts and order numbers are precisely what summarization tends to blur, and the 40-field responses show where tokens are wasted.
Production lesson: Protect facts by storing them explicitly, not by hoping a summary keeps them.

Q16 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Single answer

Correct answer: B - Acknowledge the request and escalate to a human right away, including a handoff summary of what is known so far, without first investigating.

Why the correct answer is correct: Appropriate escalation triggers include a customer's request for a human, policy exceptions or gaps, and inability to progress. When a customer explicitly demands a human, escalate immediately. The nuance is that if a customer only expresses frustration while the issue is straightforward, the agent can acknowledge and offer resolution.

Option-by-option:

A. **[INCORRECT]** Continue investigating first, because resolving the issue quickly is the best way to satisfy the customer's underlying goal, and a fast fix will probably make the request for a person unnecessary.
This overrides a clear customer preference. Offering resolution first is only appropriate when the customer did not demand a human.

B. **[CORRECT]** Acknowledge the request and escalate to a human right away, including a handoff summary of what is known so far, without first investigating.

Correct. An explicit request for a human should be honored immediately.

C. **[INCORRECT]** Run sentiment analysis and escalate only if the customer's frustration score passes a threshold.
Sentiment is not a reliable proxy for the need to escalate, and here the customer already stated what they want.

D. **[INCORRECT]** Ask the customer to explain why they want a human, and escalate only if the reason shows the case is complex.
Requiring a justification adds friction to an explicit request.

Key concept: Escalation triggers and honoring explicit customer requests (Task 5.2).
Exam reasoning: The phrase 'I want a human agent now' is an explicit request, which differs from simple frustration. That makes immediate escalation correct.
Production lesson: Respect explicit channel preferences. Reserve 'try to fix it first' for cases where the customer has not asked otherwise.

Q17 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.1 | Foundational | Single answer, secondary: D1

Correct answer: A - Send the complete conversation history on every request, including prior assistant tool_use blocks and the matching tool_result blocks.

Why the correct answer is correct: The Messages API does not remember earlier requests. Maintaining conversational coherence requires passing the complete history, including tool_use and tool_result blocks, in each request. A subagent is unnecessary; the problem is missing context, not missing agents.

Option-by-option:

A. **[CORRECT]** Send the complete conversation history on every request, including prior assistant tool_use blocks and the matching tool_result blocks.
Correct. The API is stateless, so coherence depends on passing full history.

B. **[INCORRECT]** Add a memory subagent that stores the conversation and answers the main agent's questions about what was said earlier.
This adds an agent boundary and handoff risk to solve a problem that sending the history solves directly.

C. **[INCORRECT]** Increase the length of the one-line summary so it includes more of what the customer said in earlier turns.
A hand-written summary is lossy and still omits tool results the agent needs for coherent reasoning.

D. **[INCORRECT]** Instruct the agent in the system prompt to always ask the customer to repeat details so nothing is assumed.
This makes the experience worse and does not give the model the information it needs.

Key concept: Complete conversation history in subsequent requests (Task 5.1, 1.1).
Exam reasoning: Repeated questions and duplicate lookups are classic symptoms of the model not seeing prior turns.
Production lesson: Do not add agents to compensate for missing state. Fix the state first.

Q18 | Domain 1: Agentic Architecture & Orchestration | Scenario 2: Code Generation with Claude Code

Task statement 1.6 | Intermediate | Single answer

Correct answer: B - Map the structure first, identify high-impact and high-risk areas, write a prioritized plan, and adapt the plan as dependencies are discovered.

Why the correct answer is correct: For open-ended tasks, dynamic decomposition works better than a fixed pipeline: first map the codebase, identify high-impact areas, then create a prioritized plan that adapts as dependencies appear. Fixed alphabetical or monolithic approaches cannot respond to what is discovered.

Option-by-option:

A. **[INCORRECT]** Generate tests for every file in alphabetical order using the same prompt, so coverage grows predictably, and review the results in batches of twenty files as they finish.
A fixed order ignores risk and dependencies, and it spends effort on low-value files first.

B. **[CORRECT]** Map the structure first, identify high-impact and high-risk areas, write a prioritized plan, and adapt the plan as dependencies are discovered.
Correct. Open-ended tasks need an adaptive plan built from what exploration reveals.

C. **[INCORRECT]** Ask for one test suite for the whole service in a single prompt and review the result at the end.
A single enormous request overwhelms context and gives no way to prioritize or correct course.

D. **[INCORRECT]** Test only the public API listed in the README, since internal modules are covered indirectly.
The README may be outdated and internal logic with no coverage is exactly where legacy regressions hide.

Key concept: Adaptive decomposition for open-ended work (Task 1.6).

Exam reasoning: Unclear dependencies and a very large codebase mean the plan must be built from discovery, not decided up front.
Production lesson: Use prompt chaining for predictable work and adaptive plans for exploratory work.

Q19 | Domain 2: Tool Design & MCP Integration | Scenario 2: Code Generation with Claude Code

Task statement 2.5 | Intermediate | Single answer

Correct answer: B - Read the full file, produce the modified contents with only the third block changed, and write the updated file with Write.

Why the correct answer is correct: Edit relies on unique text matching. When the text is not unique and no unique anchor exists, the reliable fallback is to Read the file in full and Write the revised contents. The other approaches either cannot succeed or change too much.

Option-by-option:

A. **[INCORRECT]** Run a Bash sed command with a global substitution so every identical block is updated in one pass.
A global substitution changes all twelve blocks, and shell edits bypass the safeguards of the Edit and Write tools.

B. **[CORRECT]** Read the full file, produce the modified contents with only the third block changed, and write the updated file with Write.
Correct. Read plus Write is the documented fallback when Edit cannot find a unique anchor.

C. **[INCORRECT]** Use Grep to locate the block and Write only that fragment to the file path.
Writing a fragment overwrites the file and destroys the other eleven blocks and everything else in it.

D. **[INCORRECT]** Retry the same Edit call several times in case the matching logic picks the right block on a later attempt, since the tool sometimes resolves ties differently between calls.
The match is ambiguous by construction, so repeating it cannot succeed.

Key concept: Edit versus Read + Write (Task 2.5).
Exam reasoning: The stem rules out a unique anchor and says only one block should change, which excludes global replacements.
Production lesson: Know each tool's failure mode and its fallback.

Q20 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.5 | Intermediate | Select 2

Correct answer: A, C - Write tests for the expected behavior and edge cases first, then run them and share the failures with Claude to iterate. / Provide two or three concrete input and expected output examples, including a null value and each date format.

Why the correct answer is correct: When prose descriptions are interpreted inconsistently, concrete input/output examples and test-driven iteration are the effective techniques. Both replace ambiguity with verifiable expectations, including edge cases such as null values.

Option-by-option:

A. **[CORRECT]** Write tests for the expected behavior and edge cases first, then run them and share the failures with Claude to iterate.
Correct. Test-driven iteration gives objective, specific feedback.

B. **[INCORRECT]** Repeat the same prose instructions in capital letters so the model gives them more weight.
Emphasis does not remove ambiguity.

C. **[CORRECT]** Provide two or three concrete input and expected output examples, including a null value and each date format.
Correct. Concrete examples communicate the transformation unambiguously when prose is read inconsistently.

D. **[INCORRECT]** Ask Claude to rate its own confidence in the script and ship it if the rating is high.
Self-reported confidence does not verify behavior on nulls and date formats.

E. **[INCORRECT]** Ask Claude to rewrite the whole script from scratch each time until the output looks right.
Rewrites discard progress and give no targeted signal about what is wrong.

Key concept: Input/output examples and test-driven iteration (Task 3.5).
Exam reasoning: Three prose attempts have already failed, which signals that the description channel is the problem.
Production lesson: Show, don't tell: examples and tests beat adjectives.

Q21 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.2 | Intermediate | Single answer

Correct answer: D - Create a skill in .claude/skills/ with a SKILL.md that sets context: fork so the analysis runs in an isolated sub-agent context and returns only its result.

Why the correct answer is correct: Skills in .claude/skills/ support frontmatter such as context: fork, which runs the skill in an isolated sub-agent context so verbose output does not pollute the main session. Project-scoped skills are shared with the team through version control.

Option-by-option:

A. **[INCORRECT]** Create an ordinary command in .claude/commands/ that runs the scan and prints the full output into the main conversation.
This shares the command but keeps the verbose output in the main context, which is the complaint.

B. **[INCORRECT]** Create the command in each developer's ~/.claude/commands/ directory so everyone has the same version locally.
User-scoped commands are personal and not shared via version control, and they still run in the main context.

C. **[INCORRECT]** Add the dependency-mapping instructions to the project CLAUDE.md so they are loaded in every session.
CLAUDE.md is always loaded. It does not run a task, and it would add instructions to every session.

D. **[CORRECT]** Create a skill in .claude/skills/ with a SKILL.md that sets context: fork so the analysis runs in an isolated sub-agent context and returns only its result.
Correct. context: fork isolates verbose output from the main conversation.

Key concept: Skills with context: fork for verbose output (Task 3.2).
Exam reasoning: The complaint is output pollution of the main context, and context: fork exists for that purpose.
Production lesson: Isolate chatty workflows so only conclusions return to the main thread.

Q22 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.4 | Intermediate | Select 2

Correct answer: A, B - Choose between an in-process cache and a shared Redis cache, which have different infrastructure requirements, and implement the choice. / Migrate the HTTP client library used across about 45 files to a new library with a different API.

Why the correct answer is correct: Plan mode suits complex tasks with large-scale changes, multiple valid approaches, architectural decisions, or multi-file modifications. Direct execution is appropriate for simple, well-scoped changes with clear scope.

Option-by-option:

A. **[CORRECT]** Choose between an in-process cache and a shared Redis cache, which have different infrastructure requirements, and implement the choice.
Correct. Multiple valid approaches with different consequences call for exploration and design first.

B. **[CORRECT]** Migrate the HTTP client library used across about 45 files to a new library with a different API.
Correct. Large multi-file migrations have architectural implications and benefit from planning.

C. **[INCORRECT]** Fix a typo in a single user-facing error message.
A trivial one-line change is a direct execution task.

D. **[INCORRECT]** Add a date validation conditional to one existing function whose behavior is fully specified.
A well-scoped, single-function change does not need a planning phase.

E. **[INCORRECT]** Add a null check to one function where a stack trace pinpoints the failing line.
The scope and cause are known, so direct execution is faster and safe.

Key concept: Plan mode versus direct execution (Task 3.4).
Exam reasoning: Scale (45 files) and architectural trade-offs (two caches with different infrastructure) are the signals; the other three are single-function fixes.
Production lesson: Plan where mistakes are expensive to unwind. Execute directly where scope is clear.

Q23 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Intermediate | Single answer

Correct answer: B - Move the conventions into the project's CLAUDE.md (or .claude/CLAUDE.md), commit it, and use /memory to confirm the files are loaded.

Why the correct answer is correct: User-level CLAUDE.md applies only to that user and is not shared via version control. Team conventions belong in project-level configuration. The /memory command shows which memory files are loaded, which helps diagnose inconsistent behavior.

Option-by-option:

A. **[INCORRECT]** Ask the new developer to copy the senior developer's user-level file into their own home directory and keep it updated by hand whenever the team's rules change.
This works once but is fragile and not versioned, so it drifts.

B. **[CORRECT]** Move the conventions into the project's CLAUDE.md (or .claude/CLAUDE.md), commit it, and use /memory to confirm the files are loaded.
Correct. Project-level instructions are shared through version control and can be verified with /memory.

C. **[INCORRECT]** Put the conventions in the new developer's personal settings so Claude applies them only for that person.
This fixes one person and leaves the team inconsistent.

D. **[INCORRECT]** Add the conventions to .claude/commands/ and ask developers to invoke the command at the start of every session.
Commands are invoked on demand, so conventions would only apply when someone remembers.

Key concept: CLAUDE.md hierarchy and diagnosing scope issues (Task 3.1).
Exam reasoning: Different outcomes for different people with the same repo point to user-level versus project-level configuration.
Production lesson: If a rule should apply to everyone, it must live where everyone gets it: in the repository.

Q24 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.3 | Challenging | Select 2

Correct answer: B, C - Create .claude/rules/terraform.md with YAML frontmatter paths: ["terraform/**/*"] containing the Terraform conventions. / Create .claude/rules/testing.md with YAML frontmatter paths: ["**/*.test.*"] containing the testing conventions.

Why the correct answer is correct: Path-specific rules in .claude/rules/ with YAML frontmatter paths and glob patterns load conditionally when matching files are edited. This is better than directory CLAUDE.md files for conventions that span many directories and better than root CLAUDE.md for conventions relevant to only some files.

Option-by-option:

A. **[INCORRECT]** Put every convention into the root CLAUDE.md under headings and rely on Claude to apply the right section.
The content loads always, costs tokens, and depends on inference about what applies.

B. **[CORRECT]** Create .claude/rules/terraform.md with YAML frontmatter paths: ["terraform/**/*"] containing the Terraform conventions.
Correct. Path-scoped rules load only when working on matching files.

C. **[CORRECT]** Create .claude/rules/testing.md with YAML frontmatter paths: ["**/*.test.*"] containing the testing conventions.
Correct. A glob applies the conventions to test files wherever they live.

D. **[INCORRECT]** Place a CLAUDE.md in every directory that contains test files with the testing conventions copied into each.
Directory CLAUDE.md files are bound to their directories, so this duplicates content across dozens of locations.

E. **[INCORRECT]** Create skills for Terraform and testing and ask developers to invoke the right one before editing.
Skills are on-demand, so conventions would not be applied automatically.

Key concept: Path-scoped rules (Task 3.3).
Exam reasoning: The two requirements, automatic loading and file-type scoping across many directories, are exactly what glob-scoped rules provide.
Production lesson: Scope instructions to the files they govern to save context and avoid conflicts.

Q25 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.2 | Foundational | Single answer

Correct answer: B - Create a skill in ~/.claude/skills/ with a different name containing the stricter variant.

Why the correct answer is correct: Personal skill customization belongs in ~/.claude/skills/ and should use a different name to avoid affecting teammates or colliding with the shared skill.

Option-by-option:

A. **[INCORRECT]** Edit the project's SKILL.md and commit it so the stricter version becomes the team default.
This changes behavior for everyone, which the developer did not want.

B. **[CORRECT]** Create a skill in ~/.claude/skills/ with a different name containing the stricter variant.
Correct. A personal skill with a distinct name stays local and does not alter the shared one.

C. **[INCORRECT]** Add the stricter rules to the project CLAUDE.md so Claude applies them whenever the skill runs.
CLAUDE.md is shared, so it would also change teammates' behavior.

D. **[INCORRECT]** Edit the project's SKILL.md locally and just remember not to commit the change.
This risks accidental commits and merge conflicts on every pull.

Key concept: Skill scope and personal customization (Task 3.2).
Exam reasoning: The requirement not to affect teammates rules out every option that edits shared files.
Production lesson: Customize in user scope; share in project scope.

Q26 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.2 | Intermediate | Single answer

Correct answer: B - Frontmatter that sets allowed-tools to Read and Write only, and an argument-hint such as [endpoint-name] showing the expected argument.

Why the correct answer is correct: Skill frontmatter can configure allowed-tools to restrict tool access while the skill runs and argument-hint to guide developers to supply required parameters. Enforcement through configuration is stronger than instructions in prose.

Option-by-option:

A. **[INCORRECT]** A rule in the project CLAUDE.md forbidding Bash and deletes in all sessions.
This would restrict every workflow in the project, not just this skill.

B. **[CORRECT]** Frontmatter that sets allowed-tools to Read and Write only, and an argument-hint such as [endpoint-name] showing the expected argument.
Correct. allowed-tools limits the tools the skill is granted, and argument-hint tells the developer which argument to supply.

C. **[INCORRECT]** A line in the skill body saying 'do not use Bash or delete files' and a note to ask the user for the endpoint name when none is given, so the behavior is documented where the skill is defined.
Body text is guidance the model might not follow, while allowed-tools restricts at the tool level.

D. **[INCORRECT]** A user-scoped command in ~/.claude/commands/ that wraps the skill and lists the allowed behavior in its description.
A personal wrapper is not shared with the team and a description does not restrict tools.

Key concept: Skill frontmatter: allowed-tools and argument-hint (Task 3.2).
Exam reasoning: 'Must never run shell commands' calls for a tool-level restriction, not a request.
Production lesson: Constrain capabilities with configuration when the constraint matters.

Q27 | Domain 4: Prompt Engineering & Structured Output | Scenario 2: Code Generation with Claude Code

Task statement 4.2 | Foundational | Single answer

Correct answer: B - Include two to four examples of correctly formatted docstrings in the prompt or project instructions.

Why the correct answer is correct: Few-shot examples are the most effective technique for consistently formatted output when detailed instructions alone produce inconsistent results. Two to four targeted examples demonstrate the desired shape.

Option-by-option:

A. **[INCORRECT]** Write a longer paragraph describing the docstring format with more rules, exceptions, and a note on tone, and place it at the top of the prompt.
More prose is interpreted inconsistently, as the existing results show.

B. **[CORRECT]** Include two to four examples of correctly formatted docstrings in the prompt or project instructions.
Correct. Few-shot examples are the most effective way to get a consistent format.

C. **[INCORRECT]** Increase the sampling temperature so the model explores more formats and picks the best.
More variation works against consistency.

D. **[INCORRECT]** Generate the docstrings as usual and fix the format afterward with a regular expression script.
A regex cannot reliably fix free-form layout differences and does not address the cause.

Key concept: Few-shot examples for format consistency (Task 4.2).
Exam reasoning: A detailed description already exists and failed, so the next lever is demonstration.
Production lesson: When instructions drift, show the format.

Q28 | Domain 4: Prompt Engineering & Structured Output | Scenario 2: Code Generation with Claude Code

Task statement 4.2 | Challenging | Single answer

Correct answer: A - Replace the set with a few diverse examples that include brief reasoning about what is tested and how boundaries are mocked.

Why the correct answer is correct: Few-shot examples enable the model to generalize judgment to novel patterns, especially when they show reasoning. Exhaustive rule lists or per-shape agents match only anticipated cases.

Option-by-option:

A. **[CORRECT]** Replace the set with a few diverse examples that include brief reasoning about what is tested and how boundaries are mocked.
Correct. Examples with reasoning teach transferable judgment rather than matching known cases.

B. **[INCORRECT]** Create a dedicated subagent per function shape, each with its own examples, and route by detected shape.
This adds agent infrastructure and still fails on shapes nobody anticipated.

C. **[INCORRECT]** Provide twenty examples of the database-call case so the model has a very strong template.
Repetition of one shape narrows the model's behavior and does not help novel structures.

D. **[INCORRECT]** Write a detailed rule for each function shape, covering what to assert and what to mock, and extend the list whenever a new shape appears in the codebase.
The list can never be complete, and it matches only pre-specified cases.

Key concept: Few-shot generalization (Task 4.2).
Exam reasoning: The failure is on a shape not covered, so the solution must transfer, not enumerate.
Production lesson: Teach the principle through varied, explained examples.

Q29 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.4 | Challenging | Single answer

Correct answer: C - Record key findings (class names, call paths) in a scratchpad file and consult it for later questions, delegate verbose exploration to subagents, and use /compact when context fills.

Why the correct answer is correct: Extended sessions suffer context degradation: the model starts using generic 'typical patterns' instead of specifics. Scratchpad files persist key findings, subagents isolate verbose discovery, and /compact reduces context when it fills with discovery output.

Option-by-option:

A. **[INCORRECT]** Run /compact repeatedly and trust the generated summaries to keep all class names and call paths.
Summaries are lossy, and compaction alone does not make specific findings durable.

B. **[INCORRECT]** Switch to a model with a larger context window and continue the same session without notes.
More room does not prevent the degradation of attention to early findings in long sessions.

C. **[CORRECT]** Record key findings (class names, call paths) in a scratchpad file and consult it for later questions, delegate verbose exploration to subagents, and use /compact when context fills.
Correct. Durable notes plus isolation and compaction counter context degradation.

D. **[INCORRECT]** Add a sentence to the prompt telling Claude to be specific and not to rely on typical patterns.
The model no longer has the details in focus, so an instruction cannot restore them.

Key concept: Context degradation and scratchpad files (Task 5.4).
Exam reasoning: References to 'typical patterns' instead of discovered classes is the textbook symptom described in the guide.
Production lesson: Externalize durable knowledge. Don't rely on a very long context to retain it.

Q30 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.4 | Intermediate | Single answer

Correct answer: B - Have each agent write its state (completed, in-progress, pending files and decisions) to a known manifest and load it on restart into the agents' prompts.

Why the correct answer is correct: For crash recovery, each agent exports state to a known location, and the coordinator loads a manifest on resume and injects it into prompts. This records exactly what was completed so work is neither lost nor repeated.

Option-by-option:

A. **[INCORRECT]** Restart the entire migration from the beginning and let the tools overwrite any previously migrated files, on the assumption that re-running the transformation on a migrated file is harmless.
This wastes the completed work and may re-apply transformations to files that are already migrated.

B. **[CORRECT]** Have each agent write its state (completed, in-progress, pending files and decisions) to a known manifest and load it on restart into the agents' prompts.
Correct. A structured manifest enables crash recovery and avoids duplicating work.

C. **[INCORRECT]** Resume the old session and rely on its stored tool results to know which files were handled.
Prior tool results may be stale or incomplete after an interruption, and they do not record work precisely.

D. **[INCORRECT]** Ask Claude to infer progress by reading the git history and guessing which files look migrated.
Guessing from appearance is unreliable and may skip partially edited files.

Key concept: Structured state persistence for crash recovery (Task 5.4).
Exam reasoning: The requirement to avoid redoing or double-applying work implies precise progress tracking, which only an explicit manifest gives.
Production lesson: Make long workflows resumable by design: write down state as you go.

Q31 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.1 | Foundational | Single answer

Correct answer: D - Run the tests with output limited to failures and a summary (via reporter options), or delegate the run to a subagent, so only relevant results enter the main context.

Why the correct answer is correct: Tool results accumulate in context and can consume tokens disproportionately to their relevance. Trimming outputs to relevant parts, or isolating verbose work in a subagent, preserves context for the task.

Option-by-option:

A. **[INCORRECT]** Clear the conversation after every command so context never grows.
Clearing discards requirements and prior decisions the session still needs.

B. **[INCORRECT]** Stop running tests during the session and run them only after all code changes are complete.
That removes the feedback loop and delays discovery of regressions.

C. **[INCORRECT]** Keep running the full suite and paste all output into the conversation so nothing is missed, since Claude can only judge the suite if it sees every passing test as well as every failure.
Thousands of passing lines consume context without informing the next action.

D. **[CORRECT]** Run the tests with output limited to failures and a summary (via reporter options), or delegate the run to a subagent, so only relevant results enter the main context.
Correct. Trimming verbose tool output keeps context focused on what matters.

Key concept: Trimming verbose tool output (Task 5.1).
Exam reasoning: Mostly passing lines carry little decision value, and the symptom is lost earlier context.
Production lesson: Control what a tool returns to the model. Relevance beats completeness.

Q32 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.3 | Intermediate | Single answer

Correct answer: A - Include the complete findings from the search and analysis subagents in the synthesis subagent's prompt, with content separated from source metadata.

Why the correct answer is correct: Subagents start with isolated context. The coordinator must provide complete findings from prior agents directly in the subagent's prompt, ideally in a structured format that separates content from metadata such as source URLs and page numbers so attribution survives.

Option-by-option:

A. **[CORRECT]** Include the complete findings from the search and analysis subagents in the synthesis subagent's prompt, with content separated from source metadata.
Correct. Subagents do not inherit context, so everything they need must be passed explicitly.

B. **[INCORRECT]** Rely on the synthesis subagent to read the coordinator's conversation history, which it inherits when the Task tool spawns it, so the coordinator only gives a short instruction about the output format.
Subagents operate with isolated context and do not inherit the coordinator's history.

C. **[INCORRECT]** Have the synthesis subagent repeat the web searches itself so it has its own fresh evidence to summarize.
This duplicates work, widens the subagent's role, and gives it tools outside its specialization.

D. **[INCORRECT]** Add a shared memory store that every subagent reads automatically at startup, then tell the coordinator to stop passing results.
Subagents do not share memory between invocations by default, and removing explicit context passing makes the problem worse.

Key concept: Explicit context passing to subagents (Task 1.3).
Exam reasoning: A generic draft with no specifics is what a subagent produces when it has been given nothing but an instruction.
Production lesson: Never assume implicit context sharing between agents. Pass what is needed.

Q33 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.2 | Intermediate | Single answer

Correct answer: D - Route all inter-subagent communication through the coordinator, which receives the URL, decides whether to invoke document analysis, and handles any errors.

Why the correct answer is correct: A hub-and-spoke design has the coordinator manage all inter-subagent communication, error handling, and information routing. This gives observability and consistent recovery and prevents hidden failures.

Option-by-option:

A. **[INCORRECT]** Merge the search and document analysis subagents into one agent so no inter-agent calls are required.
This removes specialization and tool scoping, and the coordinator would still lack visibility into steps.

B. **[INCORRECT]** Keep the direct calls and have each subagent write a log line describing the call to a shared file.
Logs improve visibility but the coordinator still cannot manage errors, and responsibility becomes distributed.

C. **[INCORRECT]** Let the document analysis subagent call back the search subagent whenever it needs more context, forming a peer loop.
Peer-to-peer chains make flow harder to trace and control, which is the original problem.

D. **[CORRECT]** Route all inter-subagent communication through the coordinator, which receives the URL, decides whether to invoke document analysis, and handles any errors.
Correct. Hub-and-spoke provides observability, consistent error handling, and controlled information flow.

Key concept: Hub-and-spoke orchestration (Task 1.2).
Exam reasoning: Invisible failure and untraceable processing are consequences of bypassing the coordinator.
Production lesson: Centralize routing so you have one place to observe, log, and recover.

Q34 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.2 | Intermediate | Single answer

Correct answer: A - Have the coordinator analyze each query's complexity and dynamically choose which subagents to invoke, skipping stages that are not needed.

Why the correct answer is correct: Coordinators should analyze query requirements and dynamically select which subagents to invoke rather than always routing through the full pipeline. More agents are not always better.

Option-by-option:

A. **[CORRECT]** Have the coordinator analyze each query's complexity and dynamically choose which subagents to invoke, skipping stages that are not needed.
Correct. Adaptive selection avoids unnecessary work for simple queries.

B. **[INCORRECT]** Run every stage in parallel for all queries so that total time equals the slowest single stage.
Later stages depend on earlier outputs, and parallelizing does not remove the unnecessary work.

C. **[INCORRECT]** Keep the full pipeline but run it on a smaller, faster model for every query to reduce cost.
This lowers cost per run but still performs unneeded stages and may degrade complex research.

D. **[INCORRECT]** Add a keyword filter before the coordinator that sends any question beginning with 'what' or 'when' straight to a cached answer table that is refreshed nightly from previous research.
Keyword matching misroutes many questions and requires a table that cannot cover arbitrary facts.

Key concept: Dynamic subagent selection (Task 1.2).

Exam reasoning: Identical cost for simple and broad queries is the sign that the coordinator does not adapt.
Production lesson: Match orchestration effort to task complexity.

Q35 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.3 | Foundational | Single answer

Correct answer: D - Add "Task" (the subagent-spawning tool) to the coordinator's allowedTools so it can spawn subagents.

Why the correct answer is correct: The Task tool is how subagents are spawned, and allowedTools must include "Task" for a coordinator to invoke them. Without it, the coordinator can only use its own tools.

Option-by-option:

A. **[INCORRECT]** Add a sentence to the system prompt saying 'You must delegate all research to the subagents and never answer a query directly yourself'.
Without the Task tool in allowedTools, the coordinator cannot delegate no matter how it is instructed.

B. **[INCORRECT]** Give each subagent the full set of tools so they can be reached without a coordinator.
This does not create a way for the coordinator to spawn them and breaks tool scoping.

C. **[INCORRECT]** Rename the subagents so their names better describe their roles, then re-run.
Names do not enable invocation.

D. **[CORRECT]** Add "Task" (the subagent-spawning tool) to the coordinator's allowedTools so it can spawn subagents.
Correct. The Task tool is the mechanism for spawning subagents, and it must be allowed.

Key concept: Task tool and allowedTools configuration (Task 1.3).
Exam reasoning: No subagent invocations in the logs plus an allowedTools list without Task is decisive.
Production lesson: When delegation never occurs, check capability before prompting.

Q36 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.1 | Foundational | Single answer

Correct answer: B - End the loop when the response's stop_reason is 'end_turn', and keep executing requested tools while it is 'tool_use'.

Why the correct answer is correct: Loops should continue on stop_reason 'tool_use' and terminate on 'end_turn'. Parsing natural language, relying on arbitrary iteration caps as the primary mechanism, or checking for text content are all anti-patterns.

Option-by-option:

A. **[INCORRECT]** End the loop after three iterations in all cases so it cannot run for long.
An arbitrary cap truncates legitimate multi-step searches and does not reflect task completion.

B. **[CORRECT]** End the loop when the response's stop_reason is 'end_turn', and keep executing requested tools while it is 'tool_use'.
Correct. The API's stop_reason is the reliable control signal.

C. **[INCORRECT]** Keep the phrase check but add a second and third phrase such as 'DONE' and 'COMPLETE', so that any of them ends the loop, and remind the model in the prompt to use one.
More phrases are still natural-language parsing and still fail when the model omits or misuses them.

D. **[INCORRECT]** End the loop as soon as the assistant message contains any text content.
Text can accompany tool_use blocks, so this would end the loop prematurely.

Key concept: Loop termination by stop_reason (Task 1.1).
Exam reasoning: Both failure modes (missing phrase, phrase plus tool call) arise from using text as the signal.
Production lesson: Use structured signals for control flow, never prose.

Q37 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.7 | Intermediate | Single answer

Correct answer: C - Resume the named session and tell the agent which two documents changed, so it re-analyzes those specific files and updates the affected conclusions.

Why the correct answer is correct: When resuming after changes, inform the agent about specific changes so it can do targeted re-analysis rather than full re-exploration. Resumption is appropriate when prior context is mostly valid; fresh starts with summaries are better when most results are stale.

Option-by-option:

A. **[INCORRECT]** Resume the session without mentioning the change, since the agent will notice changes in its tools automatically.

The agent has no way to know the documents changed unless it is told or re-reads them.

B. **[INCORRECT]** Delete the session and re-run the whole investigation from the beginning to be safe.
This wastes work when only two documents changed.

C. **[CORRECT]** Resume the named session and tell the agent which two documents changed, so it re-analyzes those specific files and updates the affected conclusions.
Correct. Prior context is mostly valid, so targeted re-analysis is efficient.

D. **[INCORRECT]** Fork the session and let the forked branch assume the original conclusions remain correct.
A fork keeps the same stale conclusions and offers no benefit here.

Key concept: Resuming sessions after changes (Task 1.7).
Exam reasoning: Only two documents changed, so most of the history remains valid and a targeted update is the efficient choice.
Production lesson: Tell resumed agents what changed instead of making them rediscover it.

Q38 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.3 | Challenging | Single answer

Correct answer: A - Emit multiple Task calls in one coordinator response for the independent subagents, and rewrite the prompt to state research goals and quality criteria instead of numbered steps.

Why the correct answer is correct: Parallel subagents are spawned by emitting multiple Task tool calls in one coordinator response. Coordinator prompts should specify goals and quality criteria rather than step-by-step procedures so subagents can adapt.

Option-by-option:

A. **[CORRECT]** Emit multiple Task calls in one coordinator response for the independent subagents, and rewrite the prompt to state research goals and quality criteria instead of numbered steps.
Correct. Parallel Task calls reduce latency and goal-based prompts allow subagents to adapt.

B. **[INCORRECT]** Have the web search subagent spawn the document analysis subagent as soon as it finds a file, and shorten the prompt.
Subagent-to-subagent spawning bypasses the coordinator and fails to address the procedural prompt.

C. **[INCORRECT]** Run both subagents from application code in two parallel threads, and keep the numbered steps in the prompt so each subagent's behavior stays predictable and easy to audit across runs.
Moving orchestration out of the model removes coordinator judgment and keeps the rigid steps.

D. **[INCORRECT]** Replace both subagents with one agent that performs search and analysis in sequence from a long list of instructions.
This loses specialization and parallelism and makes the prompt more procedural.

Key concept: Parallel spawning and goal-oriented prompts (Task 1.3).
Exam reasoning: Independent tasks across separate turns and rigid numbered steps each map to a specific correction.
Production lesson: Parallelize independent work and describe outcomes, not procedures.

Q39 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.2 | Challenging | Single answer, secondary: D5

Correct answer: C - Separate access failures from valid empty results: return structured error context for timeouts, and a zero-match success only when the search actually ran.

Why the correct answer is correct: Access failures (timeouts needing retry decisions) and valid empty results (successful queries with no matches) mean different things. Silently converting failures to empty successes suppresses errors and leads to false claims of absence.

Option-by-option:

A. **[INCORRECT]** Have the coordinator re-run every empty search with broader terms to see if something appears.
This does not tell the coordinator whether the original query executed, and may broaden irrelevantly.

B. **[INCORRECT]** Keep empty-success responses but add a low-confidence label to the final report in every case.
A blanket label does not tell the reader or coordinator which searches actually failed.

C. **[CORRECT]** Separate access failures from valid empty results: return structured error context for timeouts, and a zero-match success only when the search actually ran.
Correct. The coordinator can retry, rephrase, or annotate the gap.

D. **[INCORRECT]** Treat every empty result as a failure and retry the search three times with exponential backoff before reporting a problem, logging each attempt for later analysis.
Genuine searches may legitimately return nothing, and retrying them wastes effort.

Key concept: Access failure versus valid empty result (Task 2.2, 5.3).
Exam reasoning: The report asserted absence of evidence from a search that never happened, which is the damage caused by hiding the failure.
Production lesson: Never let an error masquerade as an empty answer.

Q40 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.4 | Foundational | Single answer

Correct answer: D - Expose the report catalog as an MCP resource so the agent can see what is available without exploratory tool calls.

Why the correct answer is correct: MCP resources expose content catalogs such as documentation hierarchies, issue summaries, or database schemas so agents have visibility into available data without exploratory tool calls.

Option-by-option:

A. **[INCORRECT]** Tell the agent to guess likely report titles and open them directly.
Guessing wastes calls and misses reports.

B. **[INCORRECT]** Embed the full text of all reports in the system prompt so the agent already has everything.
That consumes massive context and does not scale.

C. **[INCORRECT]** Add another search tool with a similar name and a broader query syntax, so that the agent has more ways to find reports and can pick whichever works.
More overlapping tools add selection ambiguity and do not give an overview.

D. **[CORRECT]** Expose the report catalog as an MCP resource so the agent can see what is available without exploratory tool calls.
Correct. MCP resources are designed to expose content catalogs.

Key concept: MCP resources for catalogs (Task 2.4).
Exam reasoning: The agent is spending turns discovering what exists, which is what a catalog resource answers up front.
Production lesson: Use resources for 'what is available' and tools for 'do something'.

Q41 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.1 | Intermediate | Single answer

Correct answer: A - Rename the tools to purpose-specific names (for example extract_web_results and analyze_document_pdf) and rewrite descriptions with inputs, outputs, and boundaries for each.

Why the correct answer is correct: Ambiguous or overlapping tool descriptions cause misrouting. Renaming tools and writing descriptions that clearly differentiate purpose, inputs, outputs, and boundaries is the primary fix.

Option-by-option:

A. **[CORRECT]** Rename the tools to purpose-specific names (for example extract_web_results and analyze_document_pdf) and rewrite descriptions with inputs, outputs, and boundaries for each.
Correct. Clear names and differentiated descriptions eliminate the overlap that causes misrouting.

B. **[INCORRECT]** Keep the names and add a system prompt line saying 'use analyze_content for the internet and analyze_document for files'.
A prompt patch leaves ambiguous tool definitions and depends on the model following the rule.

C. **[INCORRECT]** Delete one of the tools and route all requests through the remaining one, keeping its generic description.
Consolidating without clarifying inputs, outputs, and boundaries leaves the model's selection problem unsolved.

D. **[INCORRECT]** Force the model to try analyze_content first for every request and switch to analyze_document only on an error.
Trial and error doubles calls and produces avoidable failures.

Key concept: Tool description overlap (Task 2.1).
Exam reasoning: Identical descriptions are the cause, so the fix must create differences in definitions.
Production lesson: If two tools look alike to you, they look alike to the model.

Q42 | Domain 4: Prompt Engineering & Structured Output | Scenario 3: Multi-Agent Research System

Task statement 4.6 | Foundational | Single answer

Correct answer: B - Use a second, independent Claude instance without the generator's reasoning context to review the report.

Why the correct answer is correct: A model retains its reasoning context from generation, making it less likely to question its own decisions. An independent instance without that context is more effective at catching subtle issues.

Option-by-option:

A. **[INCORRECT]** Ask the generating agent to review its own work again in the same session, with explicit instructions to take a more critical tone this time.
The same session retains its own reasoning and tends not to question its decisions.

B. **[CORRECT]** Use a second, independent Claude instance without the generator's reasoning context to review the report.
Correct. Independent review is more likely to question decisions made during generation.

C. **[INCORRECT]** Increase the length of the report so the reviewer has more material to evaluate.
Longer reports make review harder, not better.

D. **[INCORRECT]** Remove the review step and rely on the quality of the initial generation.
This removes the safeguard that caught issues.

Key concept: Independent review instances (Task 4.6).
Exam reasoning: Same-instance review approving flawed output is the stated risk.
Production lesson: Review with fresh eyes, meaning a fresh context.

Q43 | Domain 4: Prompt Engineering & Structured Output | Scenario 3: Multi-Agent Research System

Task statement 4.6 | Challenging | Single answer, secondary: D5

Correct answer: D - Run a separate verifier instance that receives only each claim and its cited excerpt, without the generator's reasoning, and returns supported or unsupported.

Why the correct answer is correct: Syntactically valid citations do not guarantee semantic support. Independent review instances without the generator's reasoning context are more effective than self-review or extended thinking at catching subtle issues.

Option-by-option:

A. **[INCORRECT]** Ask the writing agent to re-check each of its own citations using extended thinking before finalizing, and to remove any claim it is unsure about so unsupported claims drop out.
The same instance retains its generation context and is less likely to challenge itself.

B. **[INCORRECT]** Add a stricter schema rule that every claim must include at least two URLs.
More URLs make the output look better but say nothing about whether the sources support the claim.

C. **[INCORRECT]** Run the generation three times and keep only claims that appear in at least two runs.
Agreement among runs of the same model can repeat the same error and may suppress true but intermittent claims.

D. **[CORRECT]** Run a separate verifier instance that receives only each claim and its cited excerpt, without the generator's reasoning, and returns supported or unsupported.
Correct. Independent, focused verification tests semantic support.

Key concept: Independent verification and semantic versus syntactic validity (Task 4.6, 5.6).
Exam reasoning: The structural checks already pass, so the missing control must examine meaning, from an independent viewpoint.
Production lesson: Schema validity is necessary, never sufficient.

Q44 | Domain 4: Prompt Engineering & Structured Output | Scenario 3: Multi-Agent Research System

Task statement 4.6 | Intermediate | Single answer, secondary: D1

Correct answer: C - Run a focused per-document pass for local issues, then a separate integration pass that compares claims across documents.

Why the correct answer is correct: Large multi-item reviews suffer attention dilution. Splitting into per-item local analysis plus a dedicated cross-item integration pass yields consistent depth and handles cross-document issues.

Option-by-option:

A. **[INCORRECT]** Run three full-length single-pass reviews and accept only the findings that appear in at least two of them, to filter out one-off errors from any single run.
Consensus filtering can discard real issues found inconsistently and does not fix the depth problem.

B. **[INCORRECT]** Split the batch into two sets of six documents and review each set in a separate session without any later comparison.
Smaller sets help depth but remove cross-document comparison, so contradictions persist.

C. **[CORRECT]** Run a focused per-document pass for local issues, then a separate integration pass that compares claims across documents.
Correct. This avoids attention dilution and handles cross-document consistency explicitly.

D. **[INCORRECT]** Ask the reviewer to be consistent across all documents and to treat all documents with equal care.
Instruction alone does not change attention dilution.

Key concept: Multi-pass review architecture (Task 4.6, 1.6).
Exam reasoning: Uneven depth and contradictory judgments are the signature of attention dilution in one pass.
Production lesson: Local passes for depth, integration passes for relationships.

Q45 | Domain 4: Prompt Engineering & Structured Output | Scenario 3: Multi-Agent Research System

Task statement 4.2 | Challenging | Single answer

Correct answer: C - Add few-shot examples showing correct extraction from documents with different structures, including one where methodology is embedded in narrative text.

Why the correct answer is correct: Few-shot examples demonstrating extraction from varied document structures reduce empty or null extraction of fields that exist in non-standard locations.

Option-by-option:

A. **[INCORRECT]** Pre-process every paper to detect the 'Methods' heading and skip papers without one.
This excludes valid papers and does not solve the extraction problem.

B. **[INCORRECT]** Instruct the model to search harder and to avoid returning null whenever possible.
A general instruction does not teach recognition of narrative methodology.

C. **[CORRECT]** Add few-shot examples showing correct extraction from documents with different structures, including one where methodology is embedded in narrative text.
Correct. Examples of varied structures address the empty extraction caused by format differences.

D. **[INCORRECT]** Mark methodology as required in the schema so the model must always produce a value.
Forcing a required field encourages fabrication when the information is hard to find rather than teaching where to look.

Key concept: Few-shot examples for varied formats (Task 4.2).
Exam reasoning: The nulls appear only for narrative-format papers, so the model needs examples of that format.
Production lesson: When the failure correlates with format, add format-diverse examples.

Q46 | Domain 5: Context Management & Reliability | Scenario 3: Multi-Agent Research System

Task statement 5.6 | Foundational | Single answer

Correct answer: A - Require structured claim-source mappings (claim, source, excerpt) from subagents and make synthesis preserve and merge them.

Why the correct answer is correct: Source attribution is lost when findings are compressed without preserving claim-source mappings. Structured mappings must be required and carried through synthesis.

Option-by-option:

A. **[CORRECT]** Require structured claim-source mappings (claim, source, excerpt) from subagents and make synthesis preserve and merge them.
Correct. Mappings keep attribution through summarization.

B. **[INCORRECT]** Remove statistics from reports to avoid unattributed numbers.
This discards valuable content instead of preserving attribution.

C. **[INCORRECT]** Ask the summarizer to add a general line listing all the sources used by the ten findings at the end of the paragraph, in alphabetical order.
A source list does not connect individual claims to sources.

D. **[INCORRECT]** Have the report generator search the web again to rediscover where each statistic came from.
Rediscovery is unreliable and wasteful when the information existed earlier.

Key concept: Provenance preservation (Task 5.6).
Exam reasoning: The statistic lost its origin at a summarization step, so the structure must travel with the claim.
Production lesson: Treat provenance as data, not as prose.

Q47 | Domain 5: Context Management & Reliability | Scenario 3: Multi-Agent Research System

Task statement 5.3 | Challenging | Single answer

Correct answer: D - Return structured error context to the coordinator, which continues with the available findings and annotates the report with the coverage gap.

Why the correct answer is correct: Subagents should recover locally from transient failures and propagate only unresolved errors with structured context. The coordinator can proceed with partial results and annotate coverage gaps. Silent suppression and total termination are both anti-patterns.

Option-by-option:

A. **[INCORRECT]** Retry the failing search indefinitely until it succeeds before producing any output.
Unbounded retries can hang the system and may never succeed.

B. **[INCORRECT]** Silently omit that sub-topic and present the report as complete.
Silent omission misleads the reader about coverage.

C. **[INCORRECT]** Terminate the entire run and tell the user the research failed, since a report with a missing sub-topic would be incomplete and might mislead decision makers.
Failing everything for one sub-topic wastes valid work.

D. **[CORRECT]** Return structured error context to the coordinator, which continues with the available findings and annotates the report with the coverage gap.
Correct. The report remains useful and is honest about what is missing.

Key concept: Error propagation and coverage annotations (Task 5.3).
Exam reasoning: A local retry already occurred and four of five sub-topics succeeded, so partial results with disclosure is optimal.
Production lesson: Fail visibly and partially, not silently or catastrophically.

Q48 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.2 | Intermediate | Single answer

Correct answer: D - Let the coordinator assess task scope and handle small, well-scoped changes with a single agent, invoking subagents only when the task needs them.

Why the correct answer is correct: Multi-agent structure carries cost: latency, handoff loss, and extra context management. Coordinators should select subagents based on query complexity, and trivial tasks do not need the full pipeline.

Option-by-option:

A. **[INCORRECT]** Add a fifth subagent that summarizes each handoff so details are not lost between the others.
This adds another boundary and another point of loss to a problem caused by too many boundaries.

B. **[INCORRECT]** Remove the coordinator and let the four subagents message each other directly as needed.
Peer messaging reduces observability and does not remove unnecessary stages.

C. **[INCORRECT]** Keep the four-agent pipeline but run the planner and the coder in parallel and cache the explorer's output, then reconcile their results at the end to reduce elapsed time.
The coder depends on the plan, and the task still pays the cost of agents it does not need.

D. **[CORRECT]** Let the coordinator assess task scope and handle small, well-scoped changes with a single agent, invoking subagents only when the task needs them.
Correct. Delegation should follow complexity, which also removes unnecessary handoff loss.

Key concept: Single agent versus multi-agent (Task 1.2).
Exam reasoning: A one-line change taking minutes and losing details points to over-engineering.
Production lesson: Use multiple agents when isolation or parallelism buys something, not by default.

Q49 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.3 | Challenging | Single answer

Correct answer: C - Restrict it to Read, Grep, and Glob and add a narrowly scoped git_history tool that only returns read-only log output for a given path.

Why the correct answer is correct: Scope tools to the role: a read-only explorer does not need Write or unrestricted Bash. For a high-frequency cross-role need, provide a narrowly scoped tool instead of broad access.

Option-by-option:

A. **[INCORRECT]** Keep Write but add a hook that logs every write so mistakes can be reverted later.
Logging after the fact does not prevent unintended modification.

B. **[INCORRECT]** Keep Bash and Write and add the instruction 'do not modify files, only run read-only commands'.
The capability to modify files remains, and compliance with the prompt is probabilistic.

C. **[CORRECT]** Restrict it to Read, Grep, and Glob and add a narrowly scoped git_history tool that only returns read-only log output for a given path.
Correct. The agent keeps what it needs without general write or shell access.

D. **[INCORRECT]** Remove all tools and have the coordinator paste file contents and history into the subagent's prompt.
This moves the exploration burden to the coordinator and defeats the subagent's purpose.

Key concept: Least-privilege tool distribution; shell-command risk (Task 2.3).
Exam reasoning: The unintended writes show the capability exists, and the history need shows why a constrained tool is better than general Bash.
Production lesson: Replace broad capabilities with narrow, purpose-built tools.

Q50 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.5 | Challenging | Single answer

Correct answer: D - First identify all exported names and wrappers of the function (the alias, re-export, and wrapper), then Grep for each name across the codebase.

Why the correct answer is correct: Grep searches content, Glob matches paths. To trace usage across wrapper modules, first identify all exported names (aliases, re-exports, wrappers) and then search for each.

Option-by-option:

A. **[INCORRECT]** Grep only for 'fetchInvoice' with a case-insensitive flag, since the alias is probably a variation of the same word.
Case-insensitivity does not catch a differently named alias or wrapper.

B. **[INCORRECT]** Use Glob with the pattern **/*invoice* to list candidate files and assume they contain all callers.
Glob matches file names, not call sites, so callers in unrelated files are missed.

C. **[INCORRECT]** Read every file in the repository into context and search by eye for any call that looks like an invoice fetch, working through the directories one at a time.
This exhausts context and misses subtle call patterns.

D. **[CORRECT]** First identify all exported names and wrappers of the function (the alias, re-export, and wrapper), then Grep for each name across the codebase.
Correct. Tracing through wrapper modules requires searching every exported name.

Key concept: Incremental code tracing with Grep and Read (Task 2.5).
Exam reasoning: The alias and wrapper explain the missing callers, so the search must cover them.
Production lesson: Follow the names, not just the original identifier.

Q51 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.4 | Challenging | Single answer

Correct answer: C - Rewrite the MCP tool description to explain its capabilities, inputs such as JQL, outputs, and when to prefer it over text search.

Why the correct answer is correct: Enhancing MCP tool descriptions to explain capabilities and outputs in detail prevents the agent from preferring built-in tools like Grep over more capable MCP tools.

Option-by-option:

A. **[INCORRECT]** Remove Grep from the agent's allowed tools so that it has to use the MCP tool for every search it performs, whether the target is code, logs, or issues.
Grep is needed for code search elsewhere, and removing it breaks other workflows.

B. **[INCORRECT]** Replace the MCP tool with a custom wrapper script that calls the same API and is run through Bash.
This adds a layer and removes the structured tool interface.

C. **[CORRECT]** Rewrite the MCP tool description to explain its capabilities, inputs such as JQL, outputs, and when to prefer it over text search.
Correct. A richer description lets the model recognize the tool as the better fit.

D. **[INCORRECT]** Add 'always use search_issues' to CLAUDE.md for every task the agent performs.
A blanket instruction overrides judgment for unrelated tasks and does not explain when the tool is better.

Key concept: MCP tool description quality (Task 2.4, 2.1).
Exam reasoning: The one-line description gives the model no reason to prefer the MCP tool.
Production lesson: Descriptions are a product surface for the model. Invest in them.

Q52 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.4 | Intermediate | Single answer

Correct answer: C - Use plan mode and delegate discovery to the Explore subagent, which returns a summary so the main conversation keeps room for design.

Why the correct answer is correct: Plan mode allows safe exploration and design before committing to changes. The Explore subagent isolates verbose discovery output and returns summaries, preserving the main context.

Option-by-option:

A. **[INCORRECT]** Read all of the module's files into the main conversation first so that every detail is available while designing the change, and then begin editing.
This floods the main context with discovery output.

B. **[INCORRECT]** Start editing right away and learn the structure as the changes break things.
Exploring through breakage risks expensive rework in a poorly understood area.

C. **[CORRECT]** Use plan mode and delegate discovery to the Explore subagent, which returns a summary so the main conversation keeps room for design.
Correct. Plan mode plus Explore isolates verbose discovery.

D. **[INCORRECT]** Create a separate custom subagent for each source file in the module and ask the main agent to merge their outputs.
That is excessive coordination for a discovery job that one Explore subagent can summarize.

Key concept: Plan mode and Explore subagent (Task 3.4).
Exam reasoning: The twin needs, understand before changing and keep the main context clean, point to plan mode with Explore.
Production lesson: Explore in isolation, decide in the main thread.

Q53 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.1 | Intermediate | Single answer

Correct answer: B - Keep standards in separate files and use @import in each package's CLAUDE.md to include only the standards relevant to that package.

Why the correct answer is correct: The @import syntax references external files to keep CLAUDE.md modular, for example importing the standards files relevant to each package. .claude/rules/ is another option for topic-specific files.

Option-by-option:

A. **[INCORRECT]** Copy the relevant standards text into each package's CLAUDE.md so every package is self-contained.
Copying duplicates content, and the copies drift.

B. **[CORRECT]** Keep standards in separate files and use @import in each package's CLAUDE.md to include only the standards relevant to that package.
Correct. @import keeps CLAUDE.md modular and relevant.

C. **[INCORRECT]** Convert each standard into a skill and require developers to invoke the right skills for their package.
Always-needed standards should not depend on manual invocation.

D. **[INCORRECT]** Keep the single root CLAUDE.md but add headings for each package and rely on Claude to ignore irrelevant sections.
All content still loads, which wastes context and relies on inference.

Key concept: CLAUDE.md modularity with @import (Task 3.1).
Exam reasoning: Different packages need different subsets, which is what selective imports express.
Production lesson: Compose configuration from small, reusable pieces.

Q54 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.5 | Foundational | Single answer

Correct answer: B - Ask Claude to interview them first about cache invalidation, failure modes, and consistency needs before writing code.

Why the correct answer is correct: The interview pattern has Claude ask questions to surface design considerations such as cache invalidation strategies and failure modes before implementing in unfamiliar domains.

Option-by-option:

A. **[INCORRECT]** Ask Claude to implement the simplest version first and review any production incidents afterward, then iterate on the design as problems are found.
This discovers the issues only after they hurt.

B. **[CORRECT]** Ask Claude to interview them first about cache invalidation, failure modes, and consistency needs before writing code.
Correct. The interview pattern surfaces considerations the developer has not thought of.

C. **[INCORRECT]** Request three completely different caching implementations and pick the one that looks cleanest.
Without knowing the requirements, the appearance of cleanliness is not a good selector.

D. **[INCORRECT]** Tell Claude to use a well-known caching library and avoid writing custom logic.
A library does not decide invalidation or consistency requirements for the developer.

Key concept: Interview pattern (Task 3.5).
Exam reasoning: The issues were unanticipated considerations, exactly what an interview is meant to expose.

Production lesson: In unfamiliar domains, ask for questions before asking for code.

Q55 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.2 | Foundational | Single answer

Correct answer: A - Put the TypeScript rule in CLAUDE.md so it always applies, and make the release checklist a skill invoked when releasing.

Why the correct answer is correct: Skills are for on-demand, task-specific workflows. CLAUDE.md is for always-loaded universal standards.

Option-by-option:

A. **[CORRECT]** Put the TypeScript rule in CLAUDE.md so it always applies, and make the release checklist a skill invoked when releasing.
Correct. Universal standards belong in always-loaded memory and task-specific workflows in on-demand skills.

B. **[INCORRECT]** Make both skills and rely on developers to invoke them as needed.
The universal rule would be skipped whenever someone forgets to invoke it.

C. **[INCORRECT]** Put the checklist in CLAUDE.md and the TypeScript rule in a skill so the longer content is always visible.
This reverses the usage pattern of both pieces of guidance.

D. **[INCORRECT]** Put both in CLAUDE.md so everything is available in every session without anyone needing to remember to invoke a command or skill or to search for it.
A 40-step checklist loaded every time wastes context for work that is rarely needed.

Key concept: Skills versus CLAUDE.md (Task 3.2).
Exam reasoning: One rule is universal and one is rare and long, which map to different mechanisms.
Production lesson: Load always what is needed always; load on demand what is not.

Q56 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.4 | Intermediate | Single answer

Correct answer: D - Use plan mode to investigate usage and design the migration approach for review, then switch to direct execution to implement the approved plan.

Why the correct answer is correct: Plan mode suits library migrations affecting many files. Combining plan mode for investigation with direct execution for implementation is a documented pattern.

Option-by-option:

A. **[INCORRECT]** Remain in plan mode through implementation so that each edit is first proposed for approval.
Plan mode is for design. Staying in it for 47 files adds friction after the approach is settled.

B. **[INCORRECT]** Write a long manual migration guide and have developers perform the edits by hand.
This forgoes the tool's strength and risks inconsistency.

C. **[INCORRECT]** Start with direct execution and fix modules one at a time, adjusting the approach as wrappers and dependencies appear, which keeps each individual change small.
Without planning, wrappers and dependencies surface late and cause rework.

D. **[CORRECT]** Use plan mode to investigate usage and design the migration approach for review, then switch to direct execution to implement the approved plan.
Correct. It combines safe planning with efficient implementation.

Key concept: Combining plan mode and direct execution (Task 3.4).
Exam reasoning: Both phases are required: agreement first, efficient execution after.
Production lesson: Plan once, then execute.

Q57 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.2 | Intermediate | Select 2, secondary: D2

Correct answer: B, D - Add a few targeted few-shot examples showing ambiguous requests and why one tool was chosen over the plausible alternative. / Expand each tool description with input formats, example queries, and boundaries stating when to use it versus similar tools.

Why the correct answer is correct: Better tool descriptions and few-shot examples with reasoning improve selection among similar tools. Adding overlap, rigid routing, or tool_choice 'any' does not address the cause.

Option-by-option:

A. **[INCORRECT]** Add ten more overlapping search tools so the model can choose the closest match.
More overlapping tools increase ambiguity.

B. **[CORRECT]** Add a few targeted few-shot examples showing ambiguous requests and why one tool was chosen over the plausible alternative.
Correct. Reasoned examples demonstrate the distinction.

C. **[INCORRECT]** Set tool_choice to 'any' so the model has to choose a tool on each turn.
This forces a tool call but does nothing to improve which one is chosen.

D. **[CORRECT]** Expand each tool description with input formats, example queries, and boundaries stating when to use it versus similar tools.
Correct. Descriptions are the primary selection mechanism.

E. **[INCORRECT]** Add a keyword router in application code that always selects the tool before the model sees the request.
This bypasses the model's understanding and is brittle.

Key concept: Tool selection improvement (Task 2.1, 4.2).
Exam reasoning: Grep versus Glob confusion comes from insufficient boundary guidance.
Production lesson: Improve what the model sees, not what the model can choose from.

Q58 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.2 | Intermediate | Single answer

Correct answer: A - Add examples that distinguish acceptable patterns (contained, intentional singletons) from genuine issues (mutable state shared across threads), with reasoning.

Why the correct answer is correct: Few-shot examples distinguishing acceptable patterns from genuine issues reduce false positives while allowing generalization to novel patterns.

Option-by-option:

A. **[CORRECT]** Add examples that distinguish acceptable patterns (contained, intentional singletons) from genuine issues (mutable state shared across threads), with reasoning.
Correct. Contrasting examples teach the boundary and generalize.

B. **[INCORRECT]** Ask the agent to list every global variable without judgment and let developers decide.
This shifts the burden back to developers and loses the value of analysis.

C. **[INCORRECT]** Stop flagging global state entirely in all future summaries.
This also removes true positives.

D. **[INCORRECT]** Add the instruction 'Only flag global state when you are very sure it is a real problem and skip anything borderline' to the summary prompt, then review the false flags weekly.
Confidence wording is vague and does not define the boundary.

Key concept: Examples for false-positive reduction (Task 4.2, 4.1).
Exam reasoning: The agent cannot tell contained singletons from harmful shared state, which examples can teach.
Production lesson: Show both sides of the line.

Q59 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.1 | Foundational | Single answer

Correct answer: A - State explicit criteria for the explanation: entry points, side effects, external dependencies, data it mutates, and known risky areas.

Why the correct answer is correct: Explicit criteria produce more consistent output than vague instructions such as 'explain well'.

Option-by-option:

A. **[CORRECT]** State explicit criteria for the explanation: entry points, side effects, external dependencies, data it mutates, and known risky areas.
Correct. Explicit criteria define what a good explanation contains.

B. **[INCORRECT]** Request the explanation three times and choose the best by eye.
This increases cost and still has no standard.

C. **[INCORRECT]** Tell the agent to match the style of the last answer it gave to the same engineer.
The last answer may itself be inconsistent.

D. **[INCORRECT]** Ask for the explanation to be 'thorough but concise' and 'appropriate to the module's importance', so that the agent aims for a sensible middle ground in every answer.

Adjectives remain open to interpretation.

Key concept: Explicit criteria in prompts (Task 4.1).
Exam reasoning: Inconsistent structure comes from an unspecified target.
Production lesson: Specify what 'good' contains.

Q60 | Domain 5: Context Management & Reliability | Scenario 4: Developer Productivity with Claude

Task statement 5.4 | Challenging | Single answer

Correct answer: C - Each worker exports its completed units to a known manifest; on resume the coordinator loads it and delegates only the unfinished units.

Why the correct answer is correct: Structured state persistence for crash recovery: each agent exports state to a known location, and the coordinator loads a manifest on resume and injects it into prompts, so completed units are not re-delegated.

Option-by-option:

A. **[INCORRECT]** Restart all four workers from scratch and instruct each one to check carefully for existing edits and not to duplicate any change that was already made.
Care instructions cannot detect what has already been applied.

B. **[INCORRECT]** Have the coordinator resume its old session and trust its recollection of which workers finished.
The crash lost in-flight state and stale recollection is unreliable.

C. **[CORRECT]** Each worker exports its completed units to a known manifest; on resume the coordinator loads it and delegates only the unfinished units.
Correct. A manifest records finished work so recovery is not repeated.

D. **[INCORRECT]** Ask a fifth agent to scan the repository afterward and remove anything that looks duplicated.
Cleanup after the fact is error-prone and may remove legitimate code.

Key concept: Manifests and crash recovery; duplicate side effects (Task 5.4).
Exam reasoning: Duplicated edits after re-delegation show that the coordinator lacked a record of completion.
Production lesson: Make recovery idempotent by recording completion.

---

7. Domain-level performance review guide

Use your tracker. A domain below roughly 70% on a paper deserves a focused review before the next paper. Look at the task statements and concepts listed for each domain and re-read the matching sections of the official Exam Guide.

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here)

Task	Topic	Questions
1.1	Agentic loops for autonomous task execution	3, 36
1.2	Coordinator-subagent orchestration	33, 34, 48
1.3	Subagent invocation, context passing, spawning	32, 35, 38
1.4	Multi-step workflows: enforcement and handoff	2, 6
1.5	Agent SDK hooks: interception and normalization	4
1.6	Task decomposition strategies	5, 18
1.7	Session state, resumption, and forking	1, 7, 37

Lessons to carry forward: Treat tool results as a cache with a lifetime. On resume, carry forward durable case facts and re-fetch anything that can change. Gate every mutating tool on resource-level ownership checks, enforced in code or in the tool backend. Do not stop at 'the user was verified'. Never infer completion from message text or an arbitrary count. Use stop_reason, and make caps a guard that fails safely. Normalize at the integration boundary. Keep the model's input consistent and let code do code's work. Parallelize reads, serialize writes. Reconcile interacting facts before any irreversible action. Design handoffs as if the receiver has no other context, because often they do not.

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here)

Task	Topic	Questions
2.1	Tool interfaces: descriptions and boundaries	8, 41
2.2	Structured error responses for MCP tools	11, 39
2.3	Tool distribution and tool_choice	10, 49
2.4	MCP server integration (Claude Code and agents)	9, 40, 51
2.5	Built-in tools: Read, Write, Edit, Bash, Grep, Glob	19, 50

Lessons to carry forward: After any tool-selection regression, diff the system prompt as well as the tool descriptions. Share the configuration, never the secret. Use environment variables for credentials. The safest tool is the one the agent cannot call. Scope the tool set to the role. Design the error contract as an API for the agent: category, retryable, human-readable explanation. Know each tool's failure mode and its fallback. Never let an error masquerade as an empty answer.

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here)

Task	Topic	Questions
3.1	CLAUDE.md hierarchy, scoping, modularity	23, 53
3.2	Custom slash commands and skills	21, 25, 26, 55
3.3	Path-specific rules	24
3.4	Plan mode vs direct execution	22, 52, 56
3.5	Iterative refinement techniques	20, 54

Lessons to carry forward: Show, don't tell: examples and tests beat adjectives. Isolate chatty workflows so only conclusions return to the main thread. Plan where mistakes are expensive to unwind. Execute directly where scope is clear. If a rule should apply to everyone, it must live where everyone gets it: in the repository. Scope instructions to the files they govern to save context and avoid conflicts. Customize in user scope; share in project scope.

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here)

Task	Topic	Questions
4.1	Explicit criteria; reducing false positives	13, 14, 59
4.2	Few-shot prompting	12, 27, 28, 45, 57, 58

Task	Topic	Questions
4.6	Multi-instance and multi-pass review	42, 43, 44

Lessons to carry forward: Choose examples for the edges of the decision boundary, and include why, not just what. Replace adjectives like 'conservative' with testable rules and examples. Ship only categories you can stand behind. Fix or hide the rest. When instructions drift, show the format. Teach the principle through varied, explained examples. Review with fresh eyes, meaning a fresh context.

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here)

Task	Topic	Questions
5.1	Context management across long interactions	15, 17, 31
5.2	Escalation and ambiguity resolution	16
5.3	Error propagation across multi-agent systems	47
5.4	Context in large codebase exploration	29, 30, 60
5.6	Provenance and uncertainty in multi-source synthesis	46

Lessons to carry forward: Protect facts by storing them explicitly, not by hoping a summary keeps them. Respect explicit channel preferences. Reserve 'try to fix it first' for cases where the customer has not asked otherwise. Do not add agents to compensate for missing state. Fix the state first. Externalize durable knowledge. Don't rely on a very long context to retain it. Make long workflows resumable by design: write down state as you go. Control what a tool returns to the model. Relevance beats completeness.

8. Final revision checklist

Tick each item when you can explain it without looking at the key. Items are drawn from the concepts tested in this paper.

Done	Concept	First seen
[ ]	Session resumption versus fresh start with injected summary	Q1
[ ]	Programmatic enforcement of authorization before a financial side effect	Q2
[ ]	Agentic loop control flow and anti-patterns	Q3
[ ]	PostToolUse hooks for data normalization	Q4
[ ]	Task decomposition with shared context and controlled side effects	Q5
[ ]	Structured handoff protocols	Q6
[ ]	fork_session for divergent exploration; side-effect safety	Q7
[ ]	Keyword-sensitive system prompt instructions overriding tool descriptions	Q8
[ ]	MCP server scoping and credential management	Q9
[ ]	Tool distribution, scoped access, and least privilege	Q10
[ ]	Structured error responses with category and retryability	Q11
[ ]	Targeted few-shot examples for ambiguity	Q12
[ ]	Explicit criteria and examples instead of vague conservatism	Q13
[ ]	False positive rates and developer or reviewer trust	Q14
[ ]	Context preservation in long, multi-issue sessions	Q15
[ ]	Escalation triggers and honoring explicit customer requests	Q16
[ ]	Complete conversation history in subsequent requests	Q17
[ ]	Adaptive decomposition for open-ended work	Q18
[ ]	Edit versus Read + Write	Q19
[ ]	Input/output examples and test-driven iteration	Q20
[ ]	Skills with context: fork for verbose output	Q21
[ ]	Plan mode versus direct execution	Q22
[ ]	CLAUDE.md hierarchy and diagnosing scope issues	Q23
[ ]	Path-scoped rules	Q24
[ ]	Skill scope and personal customization	Q25
[ ]	Skill frontmatter: allowed-tools and argument-hint	Q26
[ ]	Few-shot examples for format consistency	Q27
[ ]	Few-shot generalization	Q28
[ ]	Context degradation and scratchpad files	Q29
[ ]	Structured state persistence for crash recovery	Q30
[ ]	Trimming verbose tool output	Q31
[ ]	Explicit context passing to subagents	Q32
[ ]	Hub-and-spoke orchestration	Q33
[ ]	Dynamic subagent selection	Q34
[ ]	Task tool and allowedTools configuration	Q35
[ ]	Loop termination by stop_reason	Q36
[ ]	Resuming sessions after changes	Q37
[ ]	Parallel spawning and goal-oriented prompts	Q38
[ ]	Access failure versus valid empty result	Q39
[ ]	MCP resources for catalogs	Q40

Done	Concept	First seen
[ ]	Tool description overlap	Q41
[ ]	Independent review instances	Q42
[ ]	Independent verification and semantic versus syntactic validity	Q43
[ ]	Multi-pass review architecture	Q44
[ ]	Few-shot examples for varied formats	Q45
[ ]	Provenance preservation	Q46
[ ]	Error propagation and coverage annotations	Q47
[ ]	Single agent versus multi-agent	Q48
[ ]	Least-privilege tool distribution; shell-command risk	Q49
[ ]	Incremental code tracing with Grep and Read	Q50
[ ]	MCP tool description quality	Q51
[ ]	Plan mode and Explore subagent	Q52
[ ]	CLAUDE.md modularity with @import	Q53
[ ]	Interview pattern	Q54
[ ]	Skills versus CLAUDE.md	Q55
[ ]	Combining plan mode and direct execution	Q56
[ ]	Tool selection improvement	Q57
[ ]	Examples for false-positive reduction	Q58
[ ]	Explicit criteria in prompts	Q59
[ ]	Manifests and crash recovery; duplicate side effects	Q60

Before the next paper: classify each wrong answer as (a) knowledge gap, (b) misread of the stem or qualifier, or (c) a plausible-distractor trap. See CCAF_Final_Revision_Guide.pdf for the method.